Automation in Construction 165 (2024) 105497

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

From classification to segmentation with explainable AI: A study on crack detection and growth monitoring

Florent Forest [a,*], Hugo Porta [b], Devis Tuia [b], Olga Fink [a]

[a] Intelligent Maintenance and Operations Systems (IMOS), EPFL, Lausanne, 1015, Switzerland
[b] Environmental Computational Science and Earth Observation (ECEO), EPFL, Sion, Switzerland

ARTICLE INFO

Dataset link: https://github.com/EPFL-IMOS/crack-explanations

Keywords:
Crack detection
Crack segmentation
Severity quantification
Growth monitoring
Explainable AI
Attribution maps
Deep learning

ABSTRACT

Monitoring surface cracks in infrastructure is crucial for structural health monitoring. Automatic visual inspection offers an effective solution, especially in hard-to-reach areas. Machine learning approaches have proven their effectiveness but typically require large annotated datasets for supervised training. Once a crack is detected, monitoring its severity often demands precise segmentation of the damage. However, pixel-level annotation of images for segmentation is labor-intensive. To mitigate this cost, one can leverage explainable artificial intelligence (XAI) to derive segmentations from the explanations of a classifier, requiring only weak image-level supervision. This paper proposes applying this methodology to segment and monitor surface cracks. We evaluate the performance of various XAI methods and examine how this approach facilitates severity quantification and growth monitoring. Results reveal that while the resulting segmentation masks may exhibit lower quality than those produced by supervised methods, they remain meaningful and enable severity monitoring, thus reducing substantial labeling costs. Code and data available at https://github.com/EPFL-IMOS/crack-explanations.

1. Introduction

Cracks may appear in various types of structures, including walls [1–3], road surfaces [4,5], bridges [6–9], tunnels [10], dams, beams [11], pipes [12,13], railway sleepers [14,15] and slabs [16], made from different materials such as masonry, concrete, brick, stone and wood. The detection and monitoring of these cracks are essential for ensuring structural safety and play a significant role in structural health monitoring. Traditional methods to detect surface cracks involved manual visual inspections on a regular basis. However, these inspections have several drawbacks, including limited availability of human resources, service interruption (e.g., closures of railway sections or bridges), inspector subjectivity, high costs, and challenges in accessing hazardous or contaminated areas. Automatic visual inspection provides a solution to overcome these issues by enabling efficient, cost-effective and safe structural health condition monitoring of surface cracks [17,18]. This can be achieved through the use of Unmanned Aerial Vehicles (UAVs), robots or other vehicles equipped with imaging or video capturing capabilities [19–21]. These technologies, in conjunction with data-driven approaches based on computer vision and machine learning, enable the automatic processing of large volumes of data, thereby making the assessment more objective. Image-based approaches can, naturally, only detect surface cracks. Cracks present deep inside the material, whose detection requires other types of sensing such as ultrasound [22] or X-ray [23], are not considered in this work.

The automated detection and segmentation of cracks in images pose challenges due to the diverse aspects of cracks, the complexity and diversity of material textures, and irregular illumination conditions. Various approaches have been proposed for crack detection, including classification approaches based on supervised machine learning [8,24] and deep learning, notably based on convolutional neural networks (CNNs) [1,2,4,9,25].

A significant concern in structural health monitoring is the development and propagation of cracks over time, which can lead to increased stress and eventual failure of the structure. Once a damage has been detected, it becomes crucial to monitor the evolution of its severity to trigger timely maintenance actions and prevent catastrophic consequences. The severity of a surface crack can be quantified by measuring its width, length or area [16,26–28]. These measurements can be derived from the binary segmentation mask of the crack through crack profiling techniques such as skeletonization, thinning, tracking, labeling, and width measurement [29]. However, achieving a precise segmentation of the damage is necessary for accurate measurements.

* Corresponding author.
E-mail addresses: florent.forest@epfl.ch (F. Forest), hugo.porta@epfl.ch (H. Porta), devis.tuia@epfl.ch (D. Tuia), olga.fink@epfl.ch (O. Fink).
URLs: https://eceo.epfl.ch (D. Tuia), https://imos.epfl.ch (O. Fink).

https://doi.org/10.1016/j.autcon.2024.105497
Received 19 September 2023; Received in revised form 15 March 2024; Accepted 31 May 2024
Available online 14 June 2024

Several approaches have been developed for segmentation, primarily based on image processing techniques [30]. These methods typically involve two steps: (1) image enhancement, including noise reduction, filtering, shading correction, etc. and (2) image binarization (thresholding) to obtain the crack segmentation [10,11,25,26,31–33]. Other approaches use Fourier or wavelet transforms, edge detection [6] or the percolation method [34]. Crack depth is another severity metric estimated by [35] using optical reflection properties. Nevertheless, image processing methods usually involve complex pipelines with multiple steps, providing handcrafted solutions for specific use cases. Moreover, these methods often struggle to handle complex cases like low contrast, distracting elements, diverse material textures and complex backgrounds.

Data-driven approaches based on supervised deep learning have demonstrated excellent performance in pixel-level semantic segmentation of cracks. Numerous approaches have leveraged CNNs [5,29,36,37], among which the popular architectures U-Net [19,27,38–40] and DeepLabv3+ [28,41]. In [42], the authors propose a fusion of saliency cues with the image within a U-Net. However, these approaches require extensive pixel-level annotations of a large number of images, which is a labor-intensive and tedious process. As an alternative approach for severity estimation, [16] proposed training a classifier to directly classify severity levels without the need for a segmentation step. However, this method requires images to be labeled beforehand based on their severity level and provides more limited information.

An important barrier in the development of deep learning-based automated crack segmentation systems is the high cost associated with pixel-level annotation of large sets of images. To circumvent this challenge, *unsupervised* (requiring no labels) and *weakly-supervised* (requiring only image-level labels) segmentation methods have received growing attention [43]. As an example of an unsupervised approach, Chow et al. [44] tackled crack segmentation through anomaly detection using an autoencoder. However, the lack of any supervisory signal hinders the performance of such approaches, as our experiments will also demonstrate. Another alternative is transfer learning, i.e. training a supervised segmentation model on publicly available datasets, and applying it to the target task. However, this requires the availability of a representative and similar enough dataset. For crack segmentation on standard materials like pavement, concrete, stone or masonry, such datasets are available [45]. However, there are scenarios where transfer learning is challenging to apply. First, structures can experience various damage types other than cracks, such as spalling, scaling, excretions, efflorescence and corrosion as described in [46], cracks and spallings in railway sleepers [15], steel corrosion and delamination [47], exposed rebar, alcalo-granulate reactions, water infiltration, moisture marks [48], etc. For all such types of damages, open-source segmentation datasets are not always available, as confirmed in the survey by [49]. Secondly, in many practical cases, the target data comes from a different domains than the available source datasets, preventing effective transfer learning. For example, when dealing with crack aspects, sizes or material textures that significantly differ from the training data, the model will not perform well. For example, [50] observed that directly transferring a crack detection model trained on a different dataset to their images of narrow cracks was a challenge due to numerous false positives. These scenarios highlight the need for alternative solutions. Therefore, we explore an alternative in this work. In this paper, we focus on cracks in particular due to the availability of data and ground truth labels that enable us to evaluate the proposed methodology and severity metrics. However, the proposed methodology is applicable to any damage type.

In our work, we focus on weakly-supervised approaches that leverage explainable artificial intelligence (XAI). Explainable AI aims at enhancing the transparency and trustworthiness of AI systems, which is crucial for safety-critical applications [51]. Numerous methods have been developed to explain the decisions made by machine learning-based systems, particularly for deep neural network models and image data. These methods differ in the types of explanation and in the techniques utilized to produce these explanations. *Feature attribution explanation* methods are indirect ways of explaining a model by calculating an importance score for each variable or feature handled by the model (such as pixels in an image) to predict the target outcome, resulting in so-called *attribution maps*. The main idea is to train a classifier to classify the damages, extract *explanations* of the classifier's decisions in the form of per-pixel attribution maps and derive segmentation masks from these maps. In other words, the principle is to *approximate segmentation masks by explanations*. The advantage of this approach is that while annotating images for segmentation is tedious, classification labels can be obtained at a fraction of the cost.

Numerous feature attribution methods exist in literature, differing in their approach to compute relevance scores, the desired properties or constraints to be satisfied, and consequently, the quality of the resulting explanations. Seibold et al. [52] proposed to use an XAI technique called Layer-wise Relevance Propagation (LRP) [53] to segment damages in magnetic tiles and sewer pipe images. However, this study focused only on one of such methods, namely the LRP technique, limiting the range of applicable network architectures (for instance, LRP cannot be used in models with skip-connections). Other explanation methods were not evaluated. Furthermore, [52] solely evaluated the resulting segmentation quality in terms of F1 score, precision and recall, but the damage severity was not further assessed, which is a major requirement in many structural health monitoring applications.

In this paper, we aim to build upon this initial study, offering several key contributions. Our main contributions include proposing a comprehensive methodology and evaluating the abilities of various explainability methods in generating high-quality segmentation masks for crack detection in masonry building walls. Additionally, we assess whether this framework enables quantification and monitoring of damage severity, such as crack width measurement, to facilitate timely decision-making. Following the XAI methods taxonomy from the literature review by Arrieta et al. [51], our focus is primarily on *post-hoc* feature attribution methods (also known as *feature relevance* methods) and *architecture modification* methods suitable for convolutional neural networks and image data. Post-hoc explainability methods aim at explaining the decisions of a given black-box model without inherent transparency. In this work, we evaluate and compare six post-hoc techniques: Input×Gradient [54], Layerwise Relevance Propagation (LRP) [53], Integrated Gradients [55], DeepLift [56], DeepLiftShap and GradientShap [57]. These included techniques are widely used by practitioners; they vary in terms of the relevance computation method and exhibit different, yet reasonable, computational runtimes. Additionally, our study includes one recent inherently explainable architecture modification method (B-cos networks [58]), and one recent post-hoc method that utilizes an auxiliary network to generate attribution maps (Neural Network Explainer [59]). For the latter, we also propose an extension specifically tailored for classification tasks in which one class represents the absence of a foreground object. This adaptation differs from its original formulation and is particularly relevant in damage classification scenarios. Importantly, all methods compared in our study generate attribution maps at the input resolution. Indeed, a high resolution is necessary to capture the thin structure of cracks. For this reason, we did not include methods producing class activation maps (CAM) at the feature level such as CAM [60] and Grad-CAM [61], as well as weakly-supervised approaches based on global pooling layers for heatmap generation [62–64].

The contributions of this work are summarized as follows:

1. We propose a comprehensive methodology and evaluate the performance of various explainable artificial intelligence (XAI) methods in generating high-quality segmentation masks for cracks in masonry building wall surfaces, without requiring pixel-level annotation of images. These masks are derived from the explanations of a classifier.

2. We extend upon the Neural Network Explainer method [59] to accommodate classification tasks specifically when one class represents the absence of a foreground object (i.e., damage-free image samples in the scenario of damage classification).
3. We propose to use damage-free images as baselines in the Integrated Gradients and DeepLift-based methods, improving the quality of their explanations.
4. We investigate the applicability of these methods in quantifying damage severity and monitoring its progression, thereby facilitating timely decision-making. To this aim, we evaluate crack severity quantification and growth monitoring abilities using severity metrics such as the number of cracks, maximum crack width and crack area.

The remainder of this paper is structured as follows. In Section 2, we introduce the proposed methodology to produce crack segmentation masks based on classifier explanations, as well as the explainable AI techniques used throughout our work. In addition, we derive an adaptation of the Neural Network Explainer method suitable for damage classification. The following Section 3 presents the data, crack classification models, and experimental settings. Section 4 reports the results of our experiments. Finally, Section 5 concludes the paper and discusses the outcomes and perspectives of this research.

2. Proposed methodology

In this section, we introduce the general methodology for generating crack segmentation masks without pixel-level segmentation labels. This involves utilizing classifier explanations of the explainable AI methods that are evaluated in this study and performing post-processing on the resulting attribution maps. Additionally, we propose an adaptation of the Neural Network Explainer method suitable for damage classification.

2.1. Overview of the proposed methodology

We propose the following methodology to generate crack segmentation masks based on classifier explanations, without the need for pixel-level annotation of ground-truth segmentation masks:

1. Collect and label a dataset of positive (containing a crack) and negative (damage-free, without any crack) image patches.
2. Train a binary classifier using the labeled training images.
3. Perform inference on unseen test images. For each positive prediction, extract attribution maps from the classifier corresponding to the positive class, using an XAI method able to extract attribution maps at input resolution.
4. Post-process the resulting attribution maps:
   (a) Binarize the continuous attributions to obtain binary masks.
   (b) Apply morphological operations to close gaps in the masks and remove noisy attributions.
5. Compute crack severity levels based on the resulting masks.

The main principle is to approximate segmentation masks by classifier explanations. In cases where a well-performing classifier can be obtained for distinguishing between damage-free and damaged image samples, the explanations provided by an XAI approach, especially the pixel-level attribution maps, become valuable for generating accurate segmentation masks of damages. These maps aim at highlighting the discriminative regions that contribute to the target class. In the context of crack detection, a pixel should contribute to the crack class if and only if it is part of a cracked region. Therefore, there is an expected correspondence between explanations and segmentations, as the attribution maps should accurately highlight the regions where cracks are present.

The proposed methodology is illustrated in Fig. 1(b) and compared to the standard supervised semantic segmentation workflow (Fig. 1(a)). The data labeling cost of step 1 is order of magnitudes smaller compared to the pixel-level labeling required for training a supervised segmentation algorithm, as it only requires image-level labels. In step 2, we apply a convolutional neural network as the classifier. It is important to note that while our study focuses on the binary case, the methodology can be easily extended to handle multi-class and multi-label classification, accommodating different types of damages that may co-occur in the same image. In the multi-class case, attribution maps can be extracted for each damage class. For step 3, we assume throughout this work that the crack segmentations are generated for unseen future images, which are represented as an independent hold-out test set. However, it is also possible to generate the segmentation masks for the training images. In practice, we observed no significant difference in the results. Therefore, we report the results solely based on the test set. After the post-processing (step 4), the resulting binary masks provide approximate segmentations of the cracks, allowing the quantification of crack severity in step 5.

2.2. Explainable AI methods

In this section, we present the explainable AI methods evaluated in our study. We have included several popular XAI methods, each employing distinct approaches to compute relevance scores. All methods are able to generate attribution maps at the input resolution, which is a requirement to capture the thin structures of cracks. Moreover, these methods have easy-to-use available implementations, and a reasonable computational runtime. We intentionally avoided methods that output lower-resolution maps at the feature level, e.g., CAM [60], Grad-CAM [61] and heatmap network-based approaches with global pooling layers [62–64]. We also omitted very computationally expensive methods in this study, such as perturbation-based approaches.

Input×Gradient [54] is one of the earliest gradient-based explanation methods. It operates by computing the gradient for each input dimension at the current input value and then multiplying it with the input itself. This process reveals the change in the output resulting from an infinitesimal change of the input, thus indicating the local importance of each input dimension. However, its effectiveness is limited to the immediate local information provided by the gradient.

Layer-wise Relevance Propagation (LRP) [53,65] is a popular technique for decomposing a decision into pixel-wise relevance scores. It utilizes a backward propagation process and relies on access to the model internals such as weights and activation functions. LRP operates backward and propagates the relevance scores from the upper to the lower layers of the neural network, using specifically designed *propagation rules* such as LRP-0, LRP-ϵ, LRP-γ, LRP-$\alpha\beta$ and the $z^{\mathcal{B}}$-rule. For the best explanation quality, the rules are adjusted for different types of layers and activations.

Integrated Gradients [55] calculates and accumulates the gradients along a straight-line path that interpolates between a reference input, called *baseline*, and the input. The method satisfies two desirable properties known as completeness (i.e., the sum of the attributions over all features equals the difference between the model's output at the input and at the baseline) and implementation invariance (i.e., two networks with different implementations but the same outputs for all inputs produce identical attributions).

The **DeepLift** method [56] computes relevance scores by comparing the network activations with a reference activation obtained on a baseline input. The contributions of each feature are computed as the differences between each neuron's activation and their reference activation, and propagated in a backward pass using a recursive algorithm similar to backpropagation. Being based on activations rather than gradients, it avoids shortcomings of gradient-based methods such as zero or discontinuous gradients. DeepLift's attribution quality is typically comparable to Integrated Gradients, but it runs significantly faster.

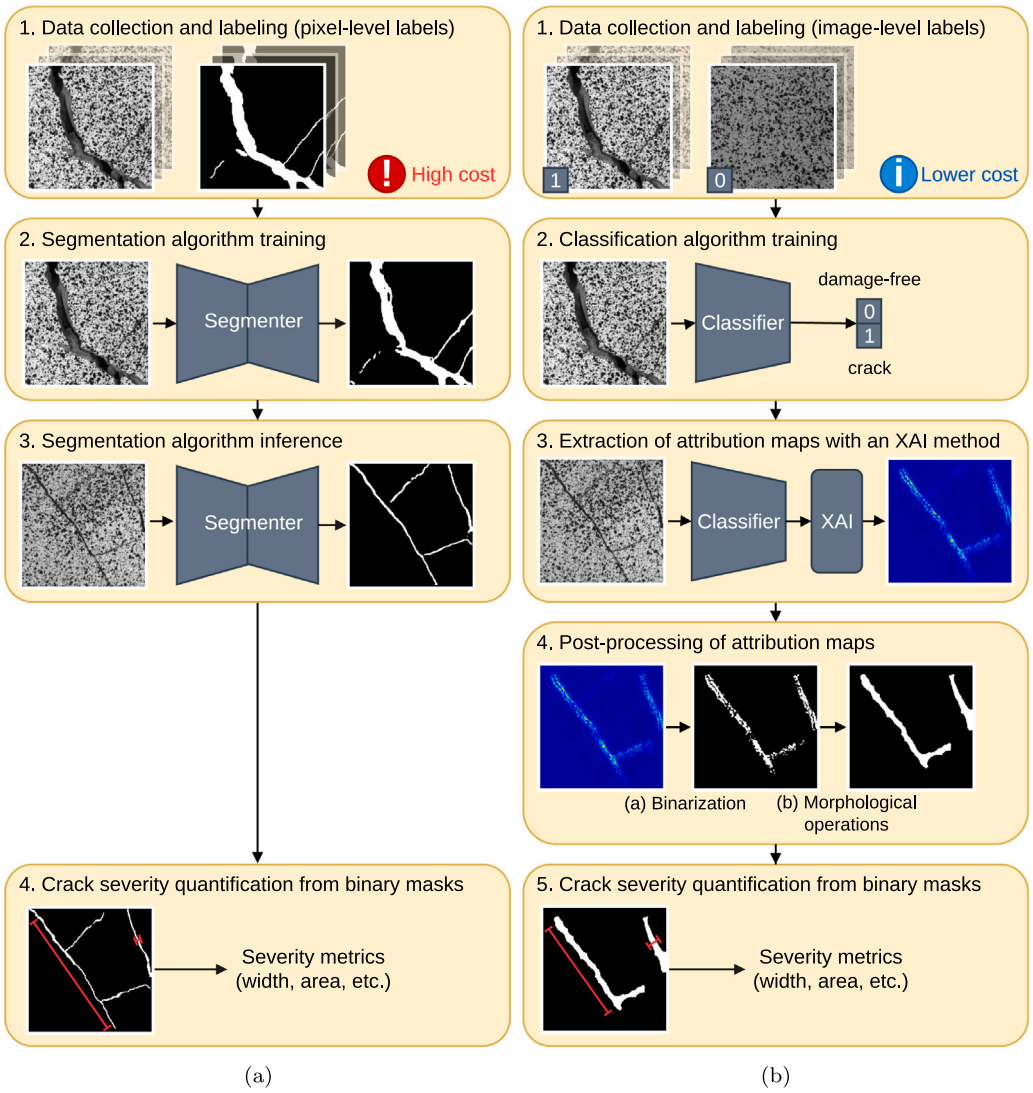

Fig. 1. Workflows for automated image-based crack segmentation and severity quantification. (a) Supervised semantic segmentation workflow. (b) Workflow of the proposed weakly-supervised methodology based on XAI and classifier explanations. This methodology allows to generate approximate segmentation masks and quantify severity while circumventing the high cost of pixel-level labeling of training images.

DeepLiftShap (also called DeepSHAP) is an application of DeepLift that uses SHAP (Shapley additive explanation) values as a measure of contribution [57]. Its attributions are estimated by sampling random images from a baseline distribution and averaging the resulting DeepLift attributions. Additionally, **GradientShap** approximates SHAP values by computing an expected gradient instead of an integral, as performed in Integrated Gradients, and can be seen as an approximation of the latter. Under model linearity and feature independence assumptions, SHAP values are approximated by the gradient expectation. The feature attributions are estimated by sampling random images from a baseline distribution and averaging their gradients multiplied by the difference between the input and the baseline.

The **Neural Network Explainer (NN-Explainer)** [59] is a recent method that trains an auxiliary network, referred to as the *explainer*, to generate attribution maps for a trained classifier, referred to as the *explanandum* (i.e., the model to be explained). These maps take the form of masks, denoted as $\mathbf{m} \in [0,1]^{W \times H}$, predicted for each class, where W and H represent the input image's width and height, respectively. Concretely, the explainer's architecture is similar to that of a segmentation network. The explainer is trained to minimize the cross-entropy of the explanandum within the image region selected by the mask and to maximize entropy (i.e., uncertainty) outside of the mask. The loss function also incorporates multiple regularization terms that penalize the area of the mask while encouraging smoothness. However, this formulation is valid for images containing one or multiple foreground objects and is not directly applicable in the context of damage classification, where one class represents the absence of foreground objects. To adapt the NN-Explainer to the context of crack detection, we propose a modification of the approach introduced in Section 2.4.

The final method considered in this study, **B-cos networks** [58], is an explainable-by-design approach that aims at making the learned model inherently transparent. The authors propose to replace all linear transformations in the network, including convolutions, with a so-called *B-cos transform*. This transform promotes alignment between weights and inputs during training, and demonstrates that the network can be faithfully summarized by a single input-dependent linear transformation. Attribution maps for a given class and input can then be obtained simply by visualizing the corresponding dimensions in the (input-dependent) matrix associated with this linear transform.

2.3. Post-processing steps

The attribution maps undergo post-processing in two stages. In the first stage, we binarize the continuous attribution maps through thresholding. In the second stage, three morphological operations are applied as follows: (1) Closing, which involves dilation followed by erosion, to

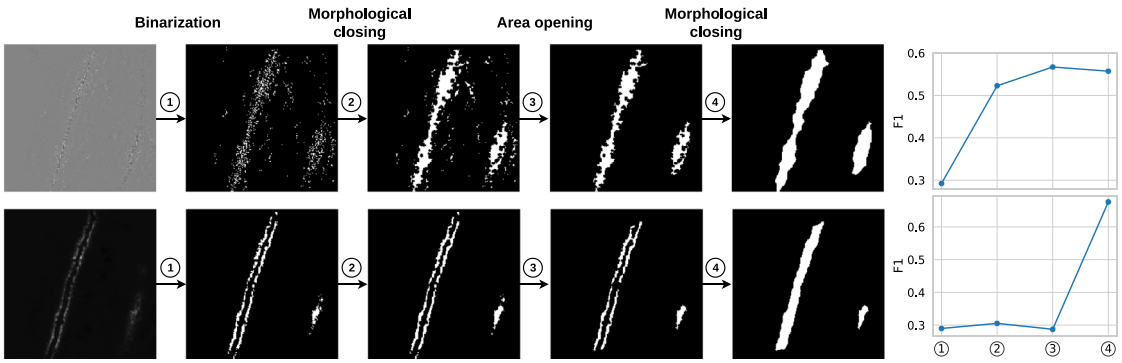

Fig. 2. Illustration of the post-processing steps using Integrated Gradients (top) and LRP (bottom) attribution maps. (1) Binarization (2) First morphological closing (3) Area opening (4) Second closing. The curve on the right shows the evolution of F1 score at each post-processing step.

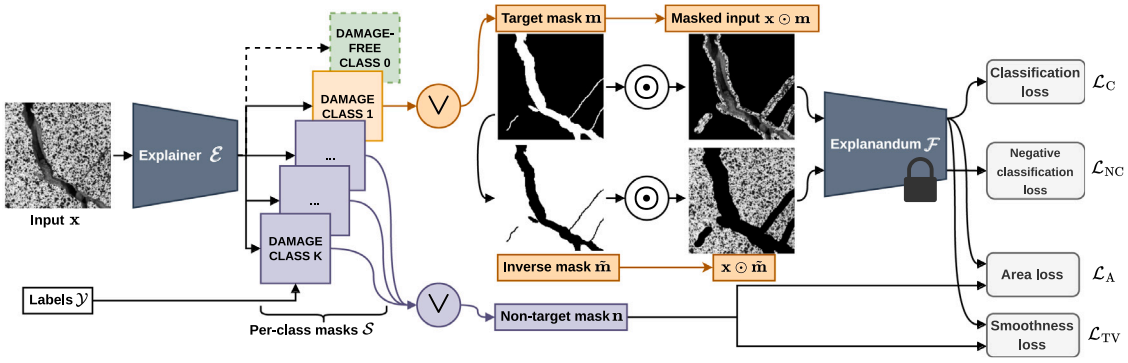

Fig. 3. Overview of the NN-Explainer method [59] for damage classification with a negative (damage-free) class and K positive (damage) classes. The *explainer* $\mathcal{E}$ learns to predict masks S for each class. Class 0 represents damage-free samples. Masks of positive classes present in the input are merged using their element-wise maximum (denoted by $\vee$ in the diagram) to create the target mask $\mathbf{m}$ (and its inverse $\tilde{\mathbf{m}}$), while masks of other positive classes that are not present form the non-target mask $\mathbf{n}$. In our application, there is a single damage class for cracks, and the non-target mask does not play a role. The *explanandum* $\mathcal{F}$ (i.e., the model to be explained) is frozen.

merge dense regions in the attribution maps; (2) Area opening with a minimum area threshold to remove remaining noise; (3) Second closing to close larger gaps in the resulting mask. Choosing an appropriate radius for the morphological closing is crucial, as a too large value will merge noisy attributions, while a too small value will result in holes in the mask. Generally, closing increases the mask area, thereby increasing recall with respect to the ground-truth segmentation. However, it may also introduce false positive pixels, thereby reducing the precision. The post-processing steps are visualized step-by-step in Fig. 2 for two different attribution methods (Integrated Gradients in the top row and LRP in the bottom row). Depending on the attribution method, different steps of the post-processing may or may not improve the segmentation quality, as measured by the F1 score relative to the ground-truth segmentation.

Please note that the quality of the segmentation could potentially be further optimized by tuning the post-processing specifically for each attribution method and final application. However, for the sake of simplicity and to ensure a fair comparison, we used identical post-processing steps across all methods in our benchmark. Furthermore, it is important to mention that the evaluated explainability techniques and the post-processing steps do not incorporate specific knowledge about the structural characteristics of cracks, such as pixel connectivity or other effective regularization techniques used in the literature [3,20,37]. While these techniques are not the focus of our study, they can be used in combination with the proposed methods to improve the resulting crack segmentation. This makes the proposed methodology general and applicable is various contexts, while enabling it to be further extended and improved for each specific application.

2.4. Adaptation of the NN-Explainer for damage classification

In this section, we derive our adaptation of the NN-Explainer [59] in the context of $(K+1)$-class multi-label classification, where K represents the number of different damage types ranging from 1 to K. In this setting, multiple damages can occur in the same image, while the negative class 0 represents the damage-free class. The illustration of our method can be found in Fig. 3. The original NN-Explainer method proposes the following loss function to train the *explainer* network:

$$\mathcal{L}_{\mathcal{E}}(\mathbf{x},\mathcal{Y},\mathbf{m},\mathbf{n}) = \mathcal{L}_{\mathrm{C}}(\mathbf{x},\mathcal{Y},\mathbf{m}) + \lambda_{\mathrm{E}}\mathcal{L}_{\mathrm{E}}(\mathbf{x},\tilde{\mathbf{m}}) + \lambda_{\mathrm{A}}\mathcal{L}_{\mathrm{A}}(\mathbf{m},\mathbf{n},S) + \lambda_{\mathrm{TV}}\mathcal{L}_{\mathrm{TV}}(\mathbf{m},\mathbf{n}) \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^{C\times W\times H}$ represents the input image (C, W, H denoting the image channels, width and height, respectively), $\mathcal{Y}$ is the set of positive target classes present in the image (defined at the image level), $\mathbf{m} \in [0,1]^{W\times H}$ is the aggregated mask generated for the target classes, $\tilde{\mathbf{m}} = 1 - \mathbf{m}$ is the inverse target mask, S is a set of per-class masks and $\mathbf{n} \in [0,1]^{W\times H}$ is the aggregated mask produced for non-target positive classes. The hyperparameters λ_{E}, λ_{A} and λ_{TV} are used for balancing the loss terms. Only positive samples containing at least one damage are used to train the explainer. In the remaining sections, we also adopt the notations from [59], where $\mathcal{E}$ represents the explainer and $\mathcal{F}$ denotes the explanandum. We now discuss the different terms of the loss function and present our proposed modification.

Classification loss: $\mathcal{L}_C(\mathbf{x},\mathcal{Y},\mathbf{m})$. This loss function encourages the target mask $\mathbf{m}$ to highlight the regions in the input that are correctly classified by the trained model. It is computed as the sum of binary cross-entropies for each positive class present in the image, using the probabilities output by $\mathcal{F}$ when applied to the masked input: $\mathbf{p} = \mathcal{F}(\mathbf{x}\odot$

$\mathbf{m}$), where $\odot$ represents element-wise multiplication. The expression for the classification loss is as follows:

$$\mathcal{L}_{\mathrm{C}}(\mathbf{x}, \mathcal{Y}, \mathbf{m}) = -\frac{1}{K}\sum_{k=1}^{K} [\![k \in \mathcal{Y}]\!] \log(\mathbf{p}[k]) + [\![k \notin \mathcal{Y}]\!] \log(1 - \mathbf{p}[k]).$$

It is important to note that in the case of single-label classification, such as in our crack detection study, this loss is equivalent to the traditional cross-entropy.

Negative entropy loss: $\mathcal{L}_E(\mathbf{x}, \tilde{\mathbf{m}})$. We propose to modify this loss term in order to adapt it for damage detection. The original formulation of NN-Explainer [59] was designed for multi-label classification tasks where every input image contains one or several objects. In such cases, where there is no specific class for the absence of objects, NN-Explainer incorporates a loss term that maximizes the classification entropy (i.e., uncertainty) when the objects of interest are masked out, representing only background to the classifier. This is achieved by computing the negative entropy of the model probabilities on the inverse-masked input $\tilde{\mathbf{p}} = \mathcal{F}(\mathbf{x} \odot \tilde{\mathbf{m}})$ across all positive classes. The formulation for the negative entropy loss is as follows:

$$\mathcal{L}_{\mathrm{E}}(\mathbf{x}, \tilde{\mathbf{m}}) = \frac{1}{K}\sum_{k=1}^{K} \tilde{\mathbf{p}}[k] \log(\tilde{\mathbf{p}}[k]). \tag{2}$$

However, in our case of damage detection, the classifier distinguishes between the presence and absence of objects (damages), with the absence (background) being represented by the negative (damage-free) class. In other words, the damage-free class is also a target class *per se*. Therefore, we replace the entropy term with the cross-entropy against the negative class, as an input containing only damage-free regions should be classified as the negative class. Thus, we propose the negative classification loss $\mathcal{L}_{\mathrm{NC}}$ as a replacement for the negative entropy loss:

$$\mathcal{L}_{\mathrm{NC}}(\mathbf{x}, \tilde{\mathbf{m}}) = -\log(\tilde{\mathbf{p}}[0]) - \frac{1}{K}\sum_{k=1}^{K} \log(1 - \tilde{\mathbf{p}}[k]). \tag{3}$$

Area loss: $\mathcal{L}_A(\mathbf{m}, \mathbf{n}, \mathbf{S})$ *and smoothness loss:* $\mathcal{L}_{TV}(\mathbf{m}, \mathbf{n})$. The area loss plays a crucial role in penalizing the size of the mask, ensuring that it remains as small as possible and preventing the trivial solution of a target mask with 1 values everywhere. It also sets constraints on the minimum and maximum allowed areas beyond which mask areas are penalized. The smoothness loss, based on the total variation, promotes smooth and artifact-free masks. No modifications have been made to these two losses from the original paper [59].

Finally, our modified loss is expressed as follows:

$$\mathcal{L}_{\mathcal{E}}(\mathbf{x}, \mathcal{Y}, \mathbf{m}, \mathbf{n}) = \mathcal{L}_{\mathrm{C}}(\mathbf{x}, \mathbf{y}, \mathbf{m}) + \lambda_{\mathrm{NC}}\mathcal{L}_{\mathrm{NC}}(\mathbf{x}, \tilde{\mathbf{m}}) + \lambda_{\mathrm{A}}\mathcal{L}_{\mathrm{A}}(\mathbf{m}, \mathbf{n}, \mathbf{S}) + \lambda_{\mathrm{TV}}\mathcal{L}_{\mathrm{TV}}(\mathbf{m}, \mathbf{n}). \tag{4}$$

3. Experiments

This section first presents the experimental data used in our experiments and the networks adopted for the crack classification task. We then provide details on the compared explainable AI (XAI) methods and the experimental settings. The following paragraphs report the results of our experiments and studies on the segmentation quality, crack severity quantification, and growth monitoring.

3.1. Data

We conducted experiments on the Experimental DIC (Digital Image Correlation) cracks dataset [3,66], which consists of 530 256 × 256 image patches from stone masonry walls that were damaged in a shear–compression loading experiment conducted at the EESD laboratory at EPFL. The mm/pixel ratio is 0.43. All the images have annotated ground-truth segmentation masks for the cracked image patches. To perform the classification, we augmented this dataset with 874 additional negative patches taken from the same walls.

Table 1
Crack classification performance on the DIC dataset (values in %).

Model	Fold	Balanced accuracy	TPR	TNR
VGG11-128	Train	99.9	100.0	99.8
	Val	96.2	95.3	97.0
	Test	89.0	79.0	99.0
VGG11-128 B-cos	Train	80.4	60.8	100.0
	Val	76.9	54.3	99.5
	Test	74.1	52.0	96.2

The training and validation sets consist of 767 patches (301 positive/466 negative) and 328 patches (129 positive/199 negative), respectively. These patches were extracted from 17 high-resolution images and randomly split. The test set, on which all results are reported, contains 309 patches (100 positive/209 negative) extracted from three different high-resolution images. Examples of images can be seen in Fig. 4. The complete dataset will be made available online, along with the code.

3.2. Crack classifier

The second step in our framework (see Fig. 1(b) in Section 2) requires training a well-performing classifier to distinguish between positive and negative image patches. We use a VGG11 [67] architecture with 128 neurons in the fully-connected layers, which we refer to as VGG11-128. We chose the VGG architecture because it is a widely-used standard CNN architecture in the related literature [3,68], and it allows us to implement LRP rules [52] as well as B-cos networks [58].

To adapt the network to our small dataset, we reduced the number of neurons in the fully-connected layers from 4096 to 128 compared to the original VGG11. We trained the VGG11-128 model from scratch, using the cross-entropy loss and the Adam optimizer with a learning rate of 10^{-4}, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and a weight decay of 10^{-8}. The only data augmentations applied are random horizontal and vertical flipping with probability 0.5. We employed early stopping based on the validation fold to select the best-performing model.

On the test set, the classifier achieved a balanced accuracy of 89%, with a true positive rate (TPR) of 79% and a true negative rate (TNR) of 99%, as reported in Table 1. While the performance is sufficient to demonstrate the approach, it is worth noting that a more advanced classifier could potentially be trained by incorporating additional data augmentations.

For the B-cos network variant (VGG11-128 B-cos), we implemented the version with MaxOut units [69] and $B = 2$, as recommended in [58]. The training parameters are identical, except for the learning rate, which is reduced to 10^{-5}. It is worth noting that the performance of the B-cos classifier model is significantly lower than that of the standard classifier model (VGG11-128). While the authors in [58] observed only a small decrease in performance on the CIFAR-10 benchmark, the performance gap is more pronounced in our task.

It is important to note that the choice of the VGG architecture does not restrict the methodology in any way. More recent architectures, such as residual networks (ResNet) and vision transformers (ViT), can also be used similarly. All the XAI methods used in our approach can be either directly applied to other architectures or with limited adaptations. In particular, gradient-based and activation-based methods can be directly applied, and an extension of LRP for Vision Transformers has been recently proposed in [70]. As we deal with a small-scale dataset throughout our study, more advanced architectures would not bring any benefit. We included additional results with different classifiers, namely ResNet-18 and ViT-B/16 (both initialized with ImageNet weights), in Appendix. To conclude, the classifier architecture shall be adapted to the data size and complexity of the task, but the overall presented methodology remains applicable.

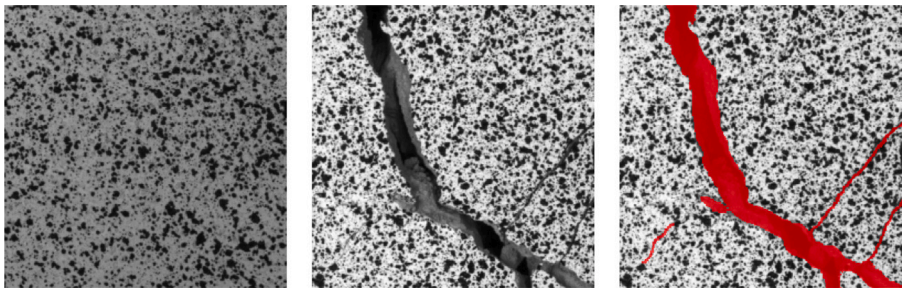

Fig. 4. (left) Example negative (damage-free) image. (middle) Example positive (cracked) image. (right) Overlay of the ground-truth crack segmentation mask in red.

3.3. Compared methods and experimental settings

In this study, we benchmark the ability of a total of eight different explainable AI methods to generate crack segmentation masks based on the pre-trained crack classifiers introduced in the previous section.

First, we evaluate six widely used post-hoc attribution methods for images, as introduced in Section 2: Input×Gradient, Integrated Gradients (IntGrad), DeepLift, DeepLiftShap, GradientShap, and Layerwise Relevance Propagation (LRP). The model to be explained is the trained VGG11-128 classifier. We used the PyTorch implementations of the `captum` library [71] for the first five methods, using the default parameters unless specified. For LRP, we adopt the segmentation rules proposed in [52]. The z^B-rule is used for the first convolutional layer, the LRP-$\alpha\beta$ rule is used for the two lower convolutional layers, the LRP-γ rule is used for all following convolutional layers, and the LRP-ϵ rule for all fully connected layers. The library `zennit` [72] was used to implement these rules in our network.

The NN-Explainer method has been adapted to the setting with a negative class for damage-free samples, as explained in Section 2.4. The explanandum is frozen and consists of the trained VGG11-128 classifier. For the explainer, a natural choice of architecture is the U-Net11 [68], as it uses an encoder similar to the one in the classifier for feature extraction. The weighting hyperparameters in the loss function are set to $\lambda_{\mathrm{NC}} = \lambda_{\mathrm{A}} = \lambda_{\mathrm{TV}} = 0.1$. The area loss also has two additional parameters to constrain the area range. We set the minimum and maximum values to 0.001 and 0.15, respectively, which roughly corresponds to the distribution of crack areas in the data, instead of the values 0.05 and 0.3 used in [59].

For the selection of class labels for the explainer training, we chose the ground-truth image-level training labels. As explained in [59], it is more suited to handle attributions for false negative predictions. In the case of significant false positives, using the explanandum's predictions as labels might be better suited, but that is not the case here. We train the explainer model from scratch, using the Adam optimizer with a learning rate of 10^{-5}, $\beta_1 = 0.9$, $\beta_2 = 0.999$ and no weight decay.

For the B-cos network, we extract the visualizations by following the procedure described in the experimental section of the paper [58].

We also include unsupervised and supervised methods, not based on XAI, for comparison.

The "Raw method" simply uses the raw pixel intensities, with the image only converted to grayscale before post-processing. We also evaluated different image enhancement techniques, such as Gaussian blurring, shading correction [31] and Min–Max Gray Level Discrimination [11], before the binarization. However, these methods did not improve the results, as they primarily address uneven illumination of the image and are unable to distinguish between the material patterns and the cracks in our dataset. As a second unsupervised comparison method, we trained a convolutional autoencoder (CAE) to reconstruct only the damage-free training images, as done in [44]. Then, the pixel-wise reconstruction error is used to generate attribution maps for the testing images. The CAE has a VGG11 encoder followed by a fully-connected layer with 128 neurons and a 100-dimensional bottleneck, and a symmetric decoder. The mean squared error loss is used to train the CAE.

Finally, as a supervised oracle method, we trained a U-Net11 [68] on the ground-truth pixel-level segmentation labels of the training set. This serves as an upper bound on the performance that could be achieved by a supervised segmentation model with access to the fully labeled dataset at pixel-level. The U-Net was trained from scratch using the Dice loss for 100 epochs, with the Adam optimizer (learning rate 10^{-4}, $\beta_1 = 0.9$, $\beta_2 = 0.999$). It is important to note that a fair comparison cannot be made between our methodology and the U-Net, as the U-Net relies on direct supervision from the pixel-level labels, which necessitates extensive annotation prior to training.

For the post-processing, we employ the *simple* or *GMM* thresholding strategies described in [52] for the binarization of attribution maps. In the second stage, we used an elliptical kernel for the morphological closing operations. The radius is set to $r = 5$ px in the first closing. The area opening operation uses a minimum area of 50 px^2. Finally, the second closing uses a radius of $r = 25$ px to close larger gaps in the masks. The parameters were fixed empirically and were kept identical across all methods in our benchmark, without specific tuning to enable a fair comparison.

3.4. Importance of the baseline image

Some relevance methods require a baseline image (such as IntGrad and DeepLift) or a baseline image distribution (such as DeepLiftShap and GradientShap) as a reference point for computing changes or gradients compared to the input. The commonly used standard baselines for these methods are a zero matrix (i.e., a black image) or a random normal baseline distribution. However, we observed that with these standard baselines the methods performed poorly, as the attributions tend to be spread over large areas surrounding the crack. To address this issue, we propose an alternative approach of sampling a set of images from the damage-free class (without cracks) as baselines. For methods like IntGrad and DeepLift, we use the mean of the sampled damage-free images as the baseline. For DeepLiftShap and GradientShap, we use these damage-free images as the baseline distribution, with 10 samples in order to keep the runtimes reasonable. This modification significantly improves the quality of attributions, as the focus of the attribution shifts more towards the crack itself rather than the healthy regions of the image. An example illustrating this improvement is visualized in Fig. 5. Quantitatively, when using the standard baseline, IntGrad obtained an F1-score of only 11.13% (after simple thresholding), whereas it increased to 18.91% when using the mean damage-free image baseline (see next paragraph for complete results). All results presented in this paper for IntGrad, DeepLift, DeepLiftShap, and GradientShap employ our proposed baseline method.

4. Results

4.1. Segmentation quality evaluation

This paragraph presents the results of our benchmark in terms of the segmentation quality of the generated masks. While localization

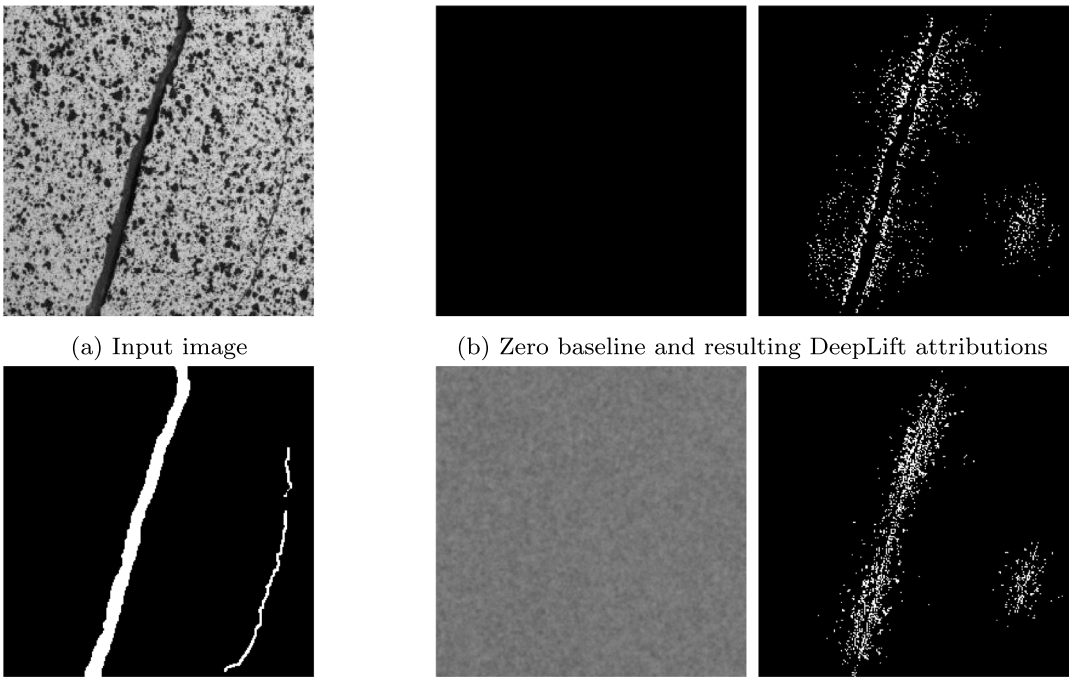

Fig. 5. Comparison between DeepLift attributions obtained with (b) the standard zero baseline and (d) our proposed baseline based on a sample of damage-free images. Attributions are more focused on the crack with our baseline. The behavior is similar for other attribution methods using a baseline image or distribution (e.g., Integrated Gradients).

performance is commonly used to evaluate attribution methods in a quantitative way, such as the grid pointing game [58,73,74], segmentation quality provides a more fine-grained measure of localization performance. However, evaluating segmentation quality requires access to ground-truth segmentation masks [52,59]. Fortunately, in our case, we have access to the true segmentation masks of the DIC images, allowing us to evaluate the segmentation quality metrics such as F1 score (also known as Dice score), Precision, Recall and Intersection-over-Union (IoU or Jaccard index) on the test set. These metrics are calculated based on the per-pixel true positives (TP), false positives (FP) and false negatives (FN) in relation to the ground-truth segmentation mask:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{F1} = \frac{2 \times \text{TP}}{2 \times \text{TP} + \text{FP} + \text{FN}} \qquad \text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

For each evaluated method, we provide the results using various combinations of thresholding and morphological post-processing, as outlined in the methodology proposed in Fig. 1(b) in Section 2. In Table 2, we present the quantitative findings. Methods are grouped by type of supervision: XAI-based (weakly-supervised, following the proposed methodology), unsupervised, and fully supervised.

The two best-performing XAI methods are DeepLiftShap and LRP, achieving F1/IoU scores of 38.1%/23.60% and 37.43%/23.03%, respectively, when used in conjunction with simple thresholding and morphological post-processing. The next two best methods are DeepLift and the B-cos network. Although these scores may appear relatively low, particularly when compared to the performance of the supervised U-Net model (83.67%/71.93%), it is important to note that these segmentations were generated without explicit supervision, and only required a trained binary classifier for cracked and non-cracked image patches. Unlike the U-Net oracle model, no pixel-level labels were necessary. The unsupervised approaches, using raw pixels and the CAE, both obtain very low performance (around 5% F1 after post-processing). In both cases, the resulting masks comprise most of the dark pixels in the images, without separating the crack from the material texture. The noisy textures of the images in our dataset, common for construction materials, hinders the autoencoder from producing accurate reconstructions. Since there is no supervisory signal, the CAE cannot separate the discriminative signal from noise and only reconstructs the mean of the image distribution. While the CAE performs well on less noisy images as shown in [44], it struggles with more complex cases like ours. In terms of post-processing, the GMM thresholding strategy generally yields the best performance. However, when morphological operations are applied, the simple strategy produces superior results. The selection of the post-processing techniques, particularly the choice of morphological operations, involves a trade-off between precision, recall and the visual aspect of the resulting mask (e.g., presence of noise or holes in the mask). Therefore, the preference for specific post-processing strategies should be based on the end user's preference and the requirements of downstream tasks.

Visualizations for five examples are shown in Fig. 6, depicting the results after binarization (with the simple thresholding strategy) in Fig. 6(a), and after the application of morphological operations in Fig. 6(b). The figure displays the image and ground-truth mask in the first two columns, followed by masks obtained by XAI methods, unsupervised methods, and the supervised U-Net. In terms of qualitative assessment, DeepLift, DeepLiftShap, and LRP produce visualizations that closely resemble the ground-truth segmentation. LRP and the B-cos network yield cleaner masks with less noise, although the attributions of the B-cos network are incomplete. Other methods exhibit more scattered and noisy attributions, resulting in masks that are too spread out. This qualitative observation corroborates with the quantitative results, showing higher recall but lower precision. The main source of error lies in the omission of very thin cracks when multiple cracks are present in the image (as observed in examples 4 and 5 in Fig. 6(b)). In such cases, the attributions of the thinner cracks are overshadowed by noise and are lost during the post-processing stage. It is worth noting that even the supervised U-Net fails to capture the two thinner cracks in the last example. The NN-Explainer method shows promising results but produces masks that are too wide, resulting in a high recall but low precision. We believe this may be due to two main reasons. Firstly, the

Table 2
Crack segmentation quality obtained with the proposed weakly-supervised methodology using different explainable AI methods and post-processing techniques (values in %). Best and second-best scores in bold underlined and bold, respectively.

	Method	Post-processing		F1	Precision	Recall	IoU
		thresh.	morph.				
XAI-based (weakly-supervised)	Input × Gradient	Simple	✗	14.64	14.44	14.85	7.90
		Simple	✓	23.30	14.37	61.55	13.19
		GMM	✗	21.54	16.17	32.28	12.07
		GMM	✓	20.76	12.41	63.52	11.58
	IntGrad	Simple	✗	18.91	26.63	14.66	10.44
		Simple	✓	27.74	20.81	41.56	16.10
		GMM	✗	28.92	25.71	33.06	16.91
		GMM	✓	25.96	17.02	54.68	14.92
	DeepLift	Simple	✗	22.58	34.97	16.67	12.73
		Simple	✓	34.44	28.75	42.96	20.81
		GMM	✗	31.70	27.06	38.27	18.84
		GMM	✓	29.55	19.81	58.12	17.33
	DeepLiftShap	Simple	✗	24.03	**47.07**	16.14	13.66
		Simple	✓	**38.19**	**36.37**	40.21	**23.60**
		GMM	✗	37.07	34.25	40.39	22.75
		GMM	✓	33.10	23.11	58.32	19.83
	GradientShap	Simple	✗	13.88	16.39	12.04	7.46
		Simple	✓	20.61	14.09	38.38	11.49
		GMM	✗	19.78	20.02	19.55	10.97
		GMM	✓	21.27	15.32	34.79	11.90
	LRP	Simple	✗	22.17	27.28	18.67	12.46
		Simple	✓	**37.43**	35.06	40.16	**23.03**
		GMM	✗	28.39	21.83	40.57	16.54
		GMM	✓	36.16	25.84	60.15	22.07
	NN-Explainer	Simple	✗	25.17	14.94	**79.80**	14.40
		Simple	✓	24.83	14.65	**81.33**	14.17
		GMM	✗	29.17	18.18	73.78	17.07
		GMM	✓	28.92	17.88	75.62	16.91
	B-cos network	Simple	✗	23.31	16.87	37.69	13.19
		Simple	✓	16.57	9.57	61.60	9.03
		GMM	✗	31.35	24.54	43.40	18.59
		GMM	✓	30.68	20.40	61.81	18.12
Unsupervised	Raw	Simple	✗	12.18	6.54	89.46	6.49
		Simple	✓	4.73	2.42	100.0	2.42
		GMM	✗	18.05	10.21	78.00	9.92
		GMM	✓	7.88	4.12	91.36	4.10
	CAE	Simple	✗	12.39	7.05	51.22	6.60
		Simple	✓	5.93	3.07	90.09	3.06
		GMM	✗	15.36	9.39	42.11	8.32
		GMM	✓	13.32	7.57	55.68	7.14
Supervised U-Net (oracle)				83.67	82.22	85.17	71.93

formulation of the explainer's training loss fails to effectively penalize the mask area, making it difficult to tightly fit around the crack. The mean total area penalty used in the loss formulation is not suitable for capturing thin, skeleton-like structures such as cracks. Secondly, masking out image regions with zero values (i.e., black pixels) may not be ideal for cracks that are comprised of dark pixels. An *in-distribution* masking operation might be more appropriate and could be explored in future work. Lastly, B-cos networks also show promising results but suffer from partial coverage of attributions over the object, failing to highlight the full length of the cracks and thereby compromising the segmentation quality. B-cos explanations are also constrained by the inferior classification performance of the B-cos network. As explained previously, the unsupervised methods are unable to separate the crack from the background texture, resulting in masks that cover the entire image.

4.2. Augmentation smoothing

Augmentation smoothing (AugSmooth) is a technique that involves averaging multiple attribution maps obtained using Test-Time Augmentations (TTA). Originally introduced with Grad-CAM [61], AugSmooth improves localization at the expense of the additional computation incurred by TTA, requiring to predict and extract attribution maps for each augmented input [75]. In our methodology, AugSmooth is applied during the generation of attribution maps, prior to post-processing. We conducted experiments using LRP+AugSmooth, and the results are presented in Table 3. For TTA, we used 6 combinations of random horizontal and vertical flipping, as well as random intensity scaling by factors 0.9, 1.0 or 1.1, following the approach described in [75]. When combined with simple thresholding, substantial improvements were observed, with an absolute increase of +2.01% in F1 score, primarily attributed to an increased recall (+4.22%). However, it is worth noting that LRP+AugSmooth slightly degraded results (−0.35%) when used in conjunction with GMM thresholding, which already exhibited high recall, due to a decrease in precision. Ultimately, when incorporating post-processing, LRP+AugSmooth achieved an F1 score approaching 40%.

4.3. Crack severity quantification

To evaluate the severity of the damage, we calculated the number of cracks per patch (CPP) [3], the total crack area per patch, and the maximum crack width. The width estimation method from [26] was used to estimate the maximum crack width. In Table 4, we report the mean absolute error (MAE) and mean absolute percentage error (MAPE, in %) of each method compared to the corresponding ground-truth metric extracted from the ground-truth mask. Results for the Raw and CAE methods have been omitted as the resulting masks do not allow meaningful estimation of crack severity. When estimating the number of CPP, most methods achieve a mean absolute error of less than 1, which is close to the performance of the U-Net supervised oracle model (0.74). The two best-performing methods are DeepLift (0.72) and DeepLiftShap (0.78), followed by LRP and NN-Explainer. However, all methods exhibit high errors in estimating crack area and width. This is primarily due to the methods overestimating the extent of the cracks or missing some cracks, as observed in Fig. 6(b). The choice of post-processing, which favors clean masks but increases their size, also contributes to this issue. Specifically, the simple thresholding is more suitable compared to the GMM strategy, which tends to overestimate the crack size. The LRP method provides the most accurate assessment of crack area and width. On average, the error is 91% for crack area, and 163% for maximum crack width, which corresponds to a 2X to 3X range. Although these errors may seem high, it is important to consider that we are dealing with very thin structures that typically cover about 1% of the image area, and mistakes of a few pixels result in a high percentage of error. In conclusion, our XAI-based methodology offers a rough estimate of crack severity metrics, but it is not as accurate as a fully supervised segmentation approach (the U-Net obtains 20% MAPE on both area and width). However, it is worth noting that for tasks involving extracting the skeleton from the binary mask, the overestimation of the crack width is not a critical issue. Furthermore, if this overestimation is consistent, results can be calibrated and still be valuable for crack growth monitoring, which we study in the following section.

4.4. Crack growth monitoring

In this section, we investigate the slightly different task of crack growth monitoring, which involves studying the evolution of crack size or severity over time. This task is equally, if not more, important, than accurately estimating severity metrics. If we observe the same damage at different time points, our methodology would be valuable if it can also highlight a potential difference in estimated severity, even if the absolute value may not be very precise. Since obtaining real images showing the growth of cracks over time is challenging, we have designed an artificial experiment as a proof-of-concept.

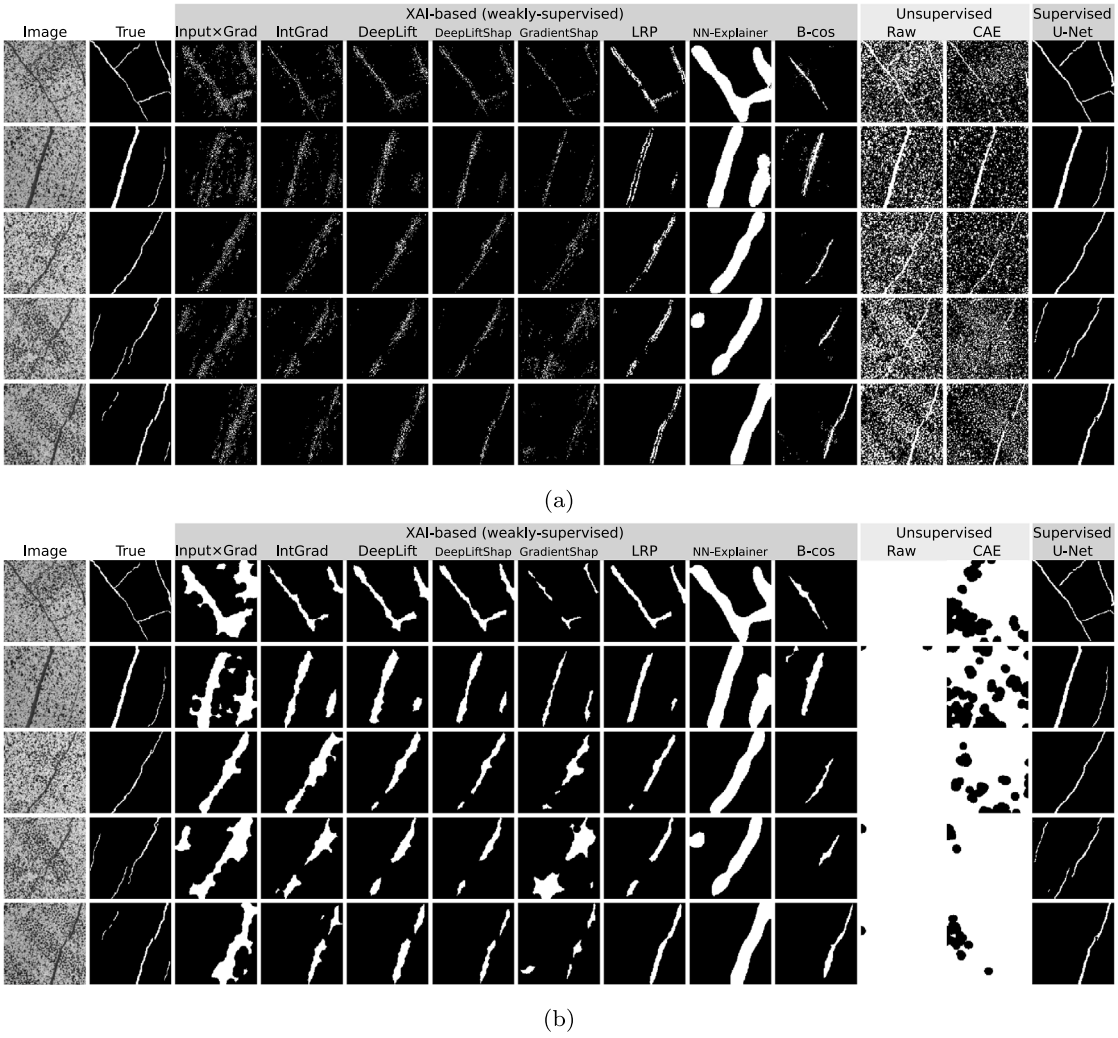

Fig. 6. Visualization of crack segmentation masks obtained through different explainable AI methods (a) after binarization of the attribution maps, and (b) after full post-processing with morphological operations. Ground-truth masks are highlighted in red on the original images. U-Net requires supervised training with pixel-level labels and serves as a reference (oracle).

Table 3
Crack segmentation quality obtained using augmentation smoothing (AugSmooth) with Test-Time Augmentations for LRP (values in %), with the delta compared to without AugSmooth. Best score in bold.

Method	Post-processing		F1	Precision	Recall	IoU
	thresh.	morph.				
LRP + AugSmooth	Simple	✗	24.21 +2.04	27.45 +0.17	21.65 +2.98	13.77 +1.31
	Simple	✓	**39.44** +2.01	**35.49** +0.43	44.38 +4.22	**24.57** +1.54
	GMM	✗	29.09 +0.70	21.41 −0.42	45.38 +4.81	17.02 +0.48
	GMM	✓	35.81 −0.35	24.91 −0.93	**63.68** +3.53	21.81 −0.26

For this experiment, we randomly sample 100 damage-free images and 100 crack masks from the DIC dataset. For each pair of samples, we simulate a linear growth trajectory by generating a sequence of five images using the original crack skeleton, growing it linearly by repeatedly applying a dilation operation (using an elliptical kernel with radius $r = 5$ px), and overlaying the grown crack onto the damage-free image. We would like to mention that this experiment does not cover the crack branching during their growth, or the initiation of new cracks. We then follow the methodology outlined in Fig. 1, using the same pre-trained classifier as in previous experiments. We derive the segmentation masks from the attribution maps of the positive class for each XAI method compared in this study. Finally, we calculate the crack areas and maximum widths based on the true and estimated masks in each growth trajectory.

One example of a generated growth trajectory is illustrated in Fig. 7. The generated images and their corresponding ground-truth masks are displayed in the first two columns. LRP explanations are displayed after thresholding (third column), and after final post-processing with morphological operations (last column). Visually, we observe that the resulting mask accurately follows the growth of the crack, except for the first image in the sequence. In this case, the crack was too thin and the classifier exhibited low confidence in its prediction (0.66 softmax probability score for the crack class), resulting in a poor corresponding explanation. For comparison, the softmax confidence scores were equal to 1.00 for the other images in the sequence.

On Fig. 8, we illustrate the evolution of true and estimated severity metrics as crack growth progresses. With the exception of image 1, the metrics consistently exhibit a monotonically increasing trend and

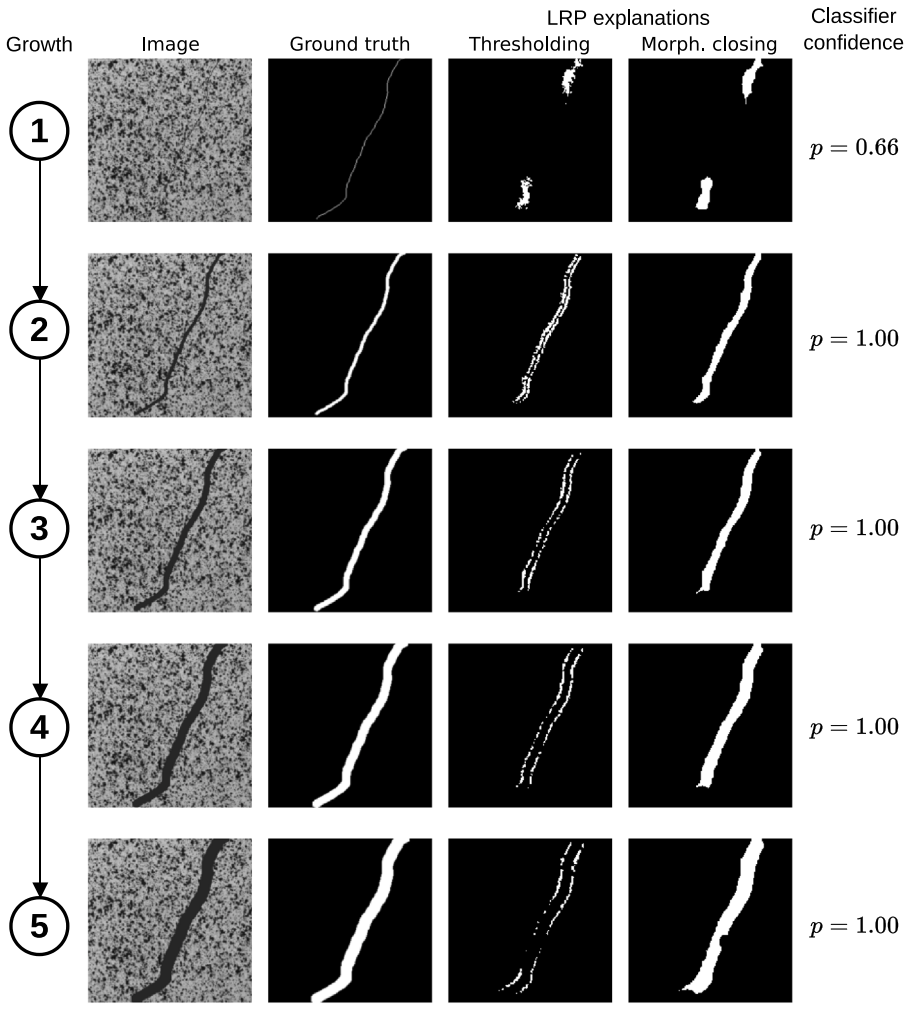

Fig. 7. Visualization of artificial linear crack growth trajectories for crack growth monitoring. The generated images and their corresponding ground-truth masks are displayed in the first two columns. LRP explanations are displayed after thresholding (third column), and after final post-processing with morphological operations (last column). In the first image, the crack is too thin and the classifier has a low prediction confidence (66% softmax score), explaining the poor corresponding explanation.

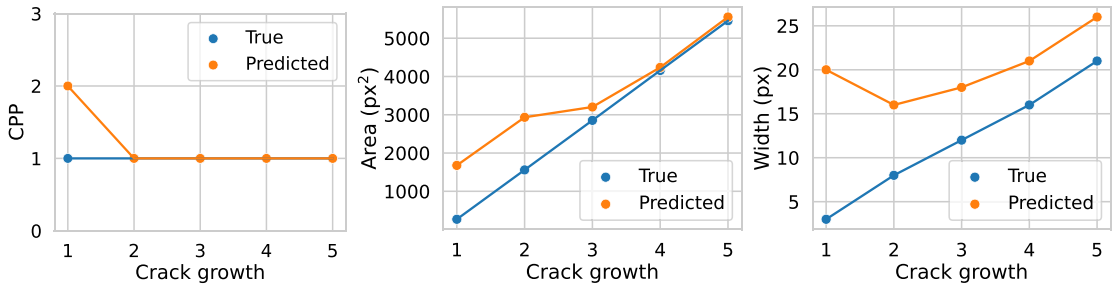

Fig. 8. Evolution of true and estimated severity metrics as a function of crack growth, using our methodology with LRP, simple thresholding and morphological closing. Even if the area and width are over-estimated, they allow to monitor the growth of the crack, as long as the classifier's explanation is relevant. The classifier exhibited low prediction confidence in the first growth step, leading to a poor explanation and severity quantification.

closely align with the true values, particularly as the crack becomes larger. While there is some overestimation in both area and width, they still provide effective means for monitoring crack growth, as long as the classifier's explanation remains relevant. Hence, it is crucial to obtain an accurate and robust classifier with good generalization abilities to ensure reliable reliance on its explanations.

To quantitatively evaluate and compare the growth monitoring abilities of the different XAI methods, we first assess whether the estimated severity metrics exhibit linear variations. This is done by computing the average r-value, representing the correlation coefficient of a linear fit of the estimated severity as a function of time. Secondly, we analyze whether the slopes of the estimated severity are in close agreement with the slopes of the ground-truth severity metric. This is assessed by calculating the mean absolute percentage error (MAPE) between the slopes. We filter the dataset to include only images classified as cracks by the classifier, and discard trajectories with fewer than three elements. This filtering process results in 87 retained growth

Table 4
Assessment of crack severity estimation using number of cracks per patch (CPP), crack area per patch and maximum crack width. The table reports the mean absolute error (MAE) or mean absolute percentage error (MAPE, in %) with the ground-truth severity measure (lower is better). Best and second-best scores in bold underlined and bold, respectively.

	Method	Post-processing		CPP	Area	Width
		thresh.	morph.	MAE	MAPE	MAPE (MAE px|mm)
XAI-based (weakly-supervised)	Input × Gradient	Simple	✓	1.13	448.1	358.8 (27.04|11.63)
		GMM	✓	1.14	629.3	404.4 (29.08|12.50)
	IntGrad	Simple	✓	0.94	271.3	268.9 (18.75|8.06)
		GMM	✓	1.10	467.3	352.3 (25.82|11.11)
	DeepLift	Simple	✓	0.81	146.0	264.6 (18.78|8.08)
		GMM	✓	**0.72**	352.1	374.2 (27.77|11.94)
	DeepLiftShap	Simple	✓	**0.78**	**103.6**	**189.2 (12.67|5.45)**
		GMM	✓	0.82	310.7	344.6 (23.84|10.25)
	GradientShap	Simple	✓	1.76	338.8	295.5 (21.52|9.25)
		GMM	✓	1.71	317.9	343.6 (24.43|10.50)
	LRP	Simple	✓	0.90	**91.0**	**163.1 (11.86|5.10)**
		GMM	✓	0.82	261.8	257.6 (18.20|7.83)
	NN-Explainer	Simple	✓	0.82	716.1	430.4 (31.41|13.51)
		GMM	✓	0.89	600.5	411.8 (29.04|12.49)
	B-cos network	Simple	✓	1.77	1325.9	195.2 (17.87|7.68)
		GMM	✓	4.04	392.4	239.7 (23.04|9.91)
Supervised U-Net (oracle)				0.74	20.1	20.8 (1.67|0.72)

Table 5
Assessment of crack growth monitoring abilities of different XAI methods in our proposed methodology, using 100 artificially generated linear growth trajectories. Severity metrics are the crack area and maximum width. The table reports the average r-value of a linear fit of the estimated severity as a function of time, and the mean absolute percentage error (MAPE, in %) between the estimated slope (i.e., growth rate) and the ground-truth slope. Best and second-best scores in bold underlined and bold, respectively.

Method	Area growth		Width growth	
	Average r	Slope MAPE	Average r	Slope MAPE
Input × Gradient	−0.32	235.9	−0.03	110.1
IntGrad	0.77	151.3	0.22	77.7
DeepLift	0.44	**84.7**	0.18	88.4
DeepLiftShap	**0.90**	88.0	**0.71**	**73.8**
GradientShap	0.33	367.7	0.07	109.1
LRP	**0.84**	**35.6**	**0.80**	**37.8**
NN-Explainer	−0.81	182.8	−0.48	120.3
B-cos network	0.71	259.7	0.28	92.4
U-Net (Oracle)	0.99	11.7	0.88	11.0

trajectories out of the initial 100. The results for the area and maximum width metrics are reported in Table 5.

We observe that only DeepLiftShap and LRP consistently exhibit a strong linear correlation for both the area and width metrics, with high r-values. IntGrad and the B-cos network achieve a high r-value for the area metric only. For the other methods, there was no clear positive correlation between crack growth and the metrics extracted from the XAI-based segmentation masks, indicating that the growth of the crack did not translate into the resulting attribution maps. However, we also observed a general degradation in the quality of the explanations for artificially generated cracks because their appearance does not closely resemble the real data distribution, which is a limitation of our study. In terms of slope, representing the severity growth rate, LRP achieves the best accuracy with an approximate 35% MAPE. The supervised oracle achieves the highest r-values and the lowest error on the growth slopes, with 11.0 and 11.7 MAPE on the area and width, respectively.

Finally, we summarize this experiment with a two-dimensional plot (Fig. 9) representing the error in severity estimation on the x-axis and the error in severity growth rate estimation (i.e., linear slope) on the y-axis, for each of the XAI methods. A good method should be located in the upper right corner, indicating low errors in both dimensions.

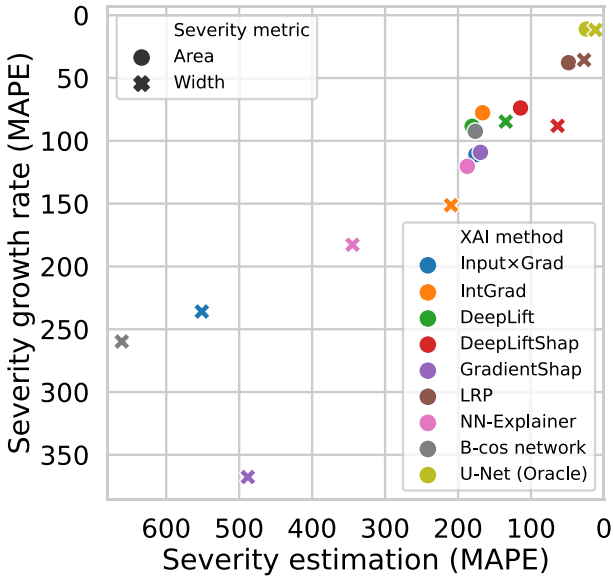

Fig. 9. Comparison of the XAI methods used in our study in terms of crack severity quantification performance and crack severity growth rate estimation performance, using area and maximum width as severity metrics. Mean absolute percentage error compared with the ground-truth severity. Best methods lie in the upper right corner. LRP is closest to the oracle method (supervised U-Net).

Among all the XAI methods compared in our study, LRP demonstrated the most consistent behavior in terms of severity quantification and growth monitoring. The second-best method is DeepLiftShap.

4.5. Runtime comparison

In this section, we compare the computational runtime performance of the evaluated methods. Experiments were performed on a server with an Intel Xeon E5-2620 CPU, an NVIDIA GeForce RTX 2080 Ti GPU and running Ubuntu 22.04. The processing times per image in seconds are reported in Fig. 10.

Among the XAI methods, the fastest is Input×Gradient at 0.251 s/image. The NN-Explainer requires a single forward pass of the *explainer* network, making it the second-fastest method at 0.664 s/image. DeepLift, GradientShap, B-cos networks and LRP each take around one second per image. DeepLiftShap requires applying DeepLift for each sample of the baseline distribution, making it naturally slower and scaling linearly with the size of the baseline distribution (here with 10 samples). Integrated Gradients is the slowest method in our study, as it requires multiple steps to interpolate between the baseline and the input. We kept the default number of steps in the `captum` library, equal to 50. The post-processing time is negligible when using the simple thresholding strategy (0.018 s/image, including the morphological operations), but increases by one order of magnitude with the GMM thresholding (0.372 s/image).

In conclusion, the LRP method provides the best compromise in terms of segmentation quality, growth monitoring ability and computational runtime. In cases where LRP cannot be used, for instance, with classifier architectures containing skip-connections, DeepLiftShap provides an effective solution. Overall, the entire approach is slower but still in a similar order of magnitude as the inference of a supervised segmentation model such as U-Net.

5. Conclusion

Automated segmentation and severity quantification of cracks in images are crucial tasks in structural health monitoring. However, deep learning-based semantic segmentation algorithms require extensive pixel-level labeling of large datasets for supervision. In this work,

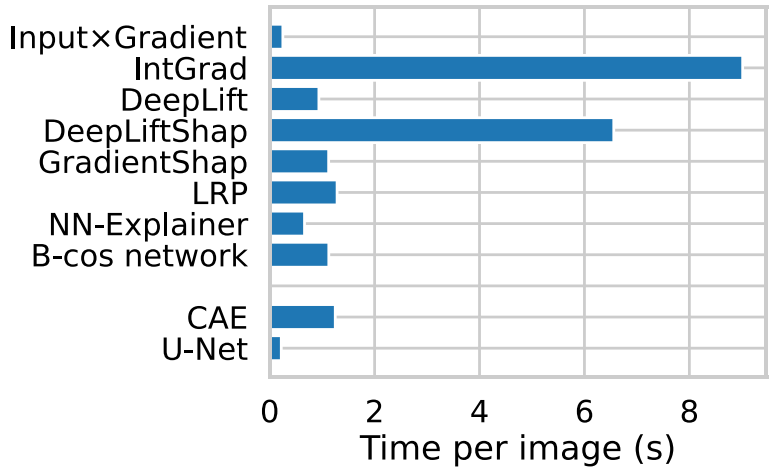

Fig. 10. Comparison of computational runtimes on the DIC images (size 256 × 256).

we have proposed a methodology and benchmarked the performance of various explainable artificial intelligence (XAI) methods, as well as post-processing techniques, in generating high-quality segmentation masks for cracks in masonry building wall surfaces. These masks are derived from the explanations of a binary classifier, trained on damage-free and damaged samples. Moreover, we have proposed a modification of the Neural Network Explainer method suitable for damage classification applications, where a negative class represents the absence of damage. Additionally, we have proposed using damage-free images as baselines in the Integrated Gradients and DeepLift-based XAI methods to enhance the quality of their explanations. The results of our benchmark study have demonstrated that this methodology allows us to approximate segmentation masks with promising performance (around 0.4 F1-score). While this performance falls below that of fully supervised segmentation approaches such as U-Net, it outperforms purely unsupervised approaches such as autoencoders.

Finally, we have investigated the applicability of these methods in quantifying damage severity and monitoring its progression. We evaluated severity metrics such as the number of cracks, maximum crack width and crack area. The experimental results demonstrated that accurately estimating the severity metrics was challenging, primarily due to overestimation of the crack extent. However, we found that it was possible to effectively monitor the severity evolution over time. One key takeaway from this study is the effectiveness of the Layer-wise Relevance Propagation (LRP) method, which excelled in terms of segmentation quality, growth monitoring ability, and computational efficiency.

It is worth mentioning that there are other variations of this methodology that can be explored. For instance, the resulting masks could serve as approximate, coarse labels for training a supervised segmentation model. If ground-truth pixel-level labels are available, these coarse labels could be fine-tuned or used in a semi-supervised setting. These strategies are left for future work.

We believe this approach holds the potential to significantly accelerate the development of automated crack detection and monitoring systems by eliminating the need for time-consuming and costly pixel-level image annotation. However, it is important to note that in scenarios where related labeled datasets are available for supervised segmentation, transfer learning is likely to produce more accurate results. Therefore, the methodology presented in this work represents an alternative in scenarios where transfer learning is challenging, e.g., when dealing with various types of damages and material aspects for which no public supervised segmentation datasets are available.

As part of future work, we also plan to apply the methodology to different types of defects and different types of infrastructure, such as railway sleepers. Moreover, we aim to evaluate the approach using real crack growth data. Finally, we intend to investigate other families of explainable AI methods, beyond feature attribution explanation methods.

CRediT authorship contribution statement

Florent Forest: Conceptualization, Data curation, Methodology, Software, Visualization, Writing – original draft, Writing – review & editing. **Hugo Porta:** Conceptualization, Software, Writing – original draft, Writing – review & editing. **Devis Tuia:** Project administration, Supervision, Writing – original draft, Writing – review & editing, Funding acquisition. **Olga Fink:** Conceptualization, Funding acquisition, Methodology, Supervision, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code and data available at https://github.com/EPFL-IMOS/crack-explanations.

Acknowledgments

This work was supported by the EPFL ENAC Cluster grant "explAIn". We also thank the EPFL EESD lab for providing additional images.

Appendix. Additional results with different architectures

In this appendix, we report additional results for different classifier architectures, namely ResNet-18 and ViT-B/16. Note that the post-processing steps have not been tuned for these models; results are not optimal and serve as a demonstration that the methodology is applicable for various classifiers beyond the VGG used in the study.

A.1. Additional results with ResNet-18 classifier

See Tables A.6–A.8

Table A.6
Crack classification performance on the DIC dataset (values in %).

Model	Fold	Balanced accuracy	TPR	TNR
ResNet-18	Train	100.0	100.0	100.0
	val	92.0	86.0	98.0
	Test	86.5	75.0	98.1

Table A.7
Crack segmentation quality obtained with the proposed weakly-supervised methodology using different explainable AI methods and post-processing techniques (values in %). Best and second-best scores in bold underlined and bold, respectively.

Method		Post-processing		F1	Precision	Recall	IoU
		thresh.	morph.				
XAI-based (weakly-supervised) – ResNet18	Input × Gradient	Simple	✗	5.01	2.79	24.53	2.57
		Simple	✓	5.87	3.03	97.16	3.03
		GMM	✗	4.19	2.93	7.33	2.14
		GMM	✓	8.43	4.77	35.99	4.40
	IntGrad	Simple	✗	9.77	5.66	35.62	5.13
		Simple	✓	7.11	3.69	**99.09**	3.68
		GMM	✗	17.35	12.86	26.67	9.50
		GMM	✓	22.86	13.68	69.64	12.91
	DeepLift	Simple	✗	15.41	10.21	31.42	8.35
		Simple	✓	11.97	6.42	88.64	6.37
		GMM	✗	26.89	**26.99**	26.79	15.53
		GMM	✓	**36.08**	26.41	56.90	**22.01**
	DeepLiftshap	Simple	✗	9.92	5.66	40.32	5.22
		Simple	✓	7.04	3.65	98.39	3.65
		GMM	✗	26.47	25.83	27.14	15.25
		GMM	✓	**37.54**	**27.04**	61.34	**23.11**
	GradientShap	Simple	✗	5.57	2.96	48.23	2.87
		Simple	✓	5.81	2.99	**98.81**	2.99
		GMM	✗	11.63	8.65	17.77	6.17
		GMM	✓	18.70	11.52	49.61	10.31
	LRP	Simple	✗	26.23	16.76	60.24	15.09
		Simple	✓	23.89	14.23	74.54	13.57
		GMM	✗	28.31	20.42	46.12	16.49
		GMM	✓	27.70	18.50	55.11	16.08
Unsupervised	Raw	Simple	✗	12.18	6.54	89.46	6.49
		Simple	✓	4.73	2.42	100.0	2.42
		GMM	✗	18.05	10.21	78.00	9.92
		GMM	✓	7.88	4.12	91.36	4.10
	CAE	Simple	✗	12.39	7.05	51.22	6.60
		Simple	✓	5.93	3.07	90.09	3.06
		GMM	✗	15.36	9.39	42.11	8.32
		GMM	✓	13.32	7.57	55.68	7.14
Supervised U-Net (oracle)				83.67	82.22	85.17	71.93

Table A.8
Assessment of crack severity estimation using number of cracks per patch (CPP), crack area per patch and maximum crack width. The table reports the mean absolute error (MAE) or mean absolute percentage error (MAPE, in %) with the ground-truth severity measure (lower is better). Best and second-best scores in bold underlined and bold, respectively.

Method		Post-processing		CPP	Area	Width
		thresh.	morph.	MAE	MAPE	MAPE (MAE px)
XAI-based – ResNet18	Input × Gradient	Simple	✓	**1.01**	5959.0	**201.0** (17.44)
		GMM	✓	3.07	1144.2	371.1 (28.29)
	IntGrad	Simple	✓	1.41	5346.8	258.8 (21.39)
		GMM	✓	2.25	916.0	361.2 (26.17)
	DeepLift	Simple	✓	1.60	2409.3	315.9 (25.27)
		GMM	✓	1.40	**367.4**	299.2 (21.40)
	DeepLiftShap	Simple	✓	1.33	5032.2	219.2 (19.27)
		GMM	✓	1.43	**350.7**	299.6 (21.56)
	GradientShap	Simple	✓	1.05	6263.1	**113.4** (12.01)
		GMM	✓	3.25	751.1	379.7 (28.28)
	LRP	Simple	✓	**0.84**	906.3	357.6 (27.37)
		GMM	✓	1.21	488.3	348.7 (26.29)
Supervised U-Net (oracle)				0.74	20.1	20.8 (1.67)

Table A.9
Crack classification performance on the DIC dataset (values in %).

Model	Fold	Balanced accuracy	TPR	TNR
ViT-B/16	Train	99.3	98.7	100.0
	val	90.7	84.5	97.0
	Test	85.8	74.0	97.6

Table A.10
Crack segmentation quality obtained with the proposed weakly-supervised methodology using different explainable AI methods and post-processing techniques (values in %). Best and second-best scores in bold underlined and bold, respectively.

Method		Post-processing		F1	Precision	Recall	IoU
		thresh.	morph.				
XAI-based (weakly-supervised) – ViT-B/16	Input × Gradient	Simple	✗	8.60	6.15	14.30	4.49
		Simple	✓	13.16	7.25	**71.06**	7.04
		GMM	✗	10.27	10.75	9.83	5.41
		GMM	✓	21.83	16.33	32.91	12.25
	IntGrad	Simple	✗	25.24	37.04	19.15	14.45
		Simple	✓	**40.81**	32.52	54.75	**25.63**
		GMM	✗	38.73	**42.91**	35.30	24.02
		GMM	✓	**46.83**	**37.16**	63.28	**30.57**
	DeepLift	Simple	✗	16.12	15.70	16.57	8.77
		Simple	✓	22.14	13.78	56.34	12.45
		GMM	✗	30.07	30.72	29.44	17.69
		GMM	✓	38.75	29.15	57.79	24.03
	DeepLiftshap	Simple	✗	14.97	14.99	14.94	8.09
		Simple	✓	22.45	14.26	52.74	12.64
		GMM	✗	27.23	28.64	25.95	15.76
		GMM	✓	37.63	28.91	53.86	23.17
	GradientShap	Simple	✗	15.71	17.79	14.07	8.52
		Simple	✓	24.20	16.14	48.37	13.77
		GMM	✗	21.11	32.35	15.66	11.80
		GMM	✓	32.35	32.77	31.94	19.29
	LRP-ViT*	Simple	✗	18.71	11.13	58.54	10.32
		Simple	✓	17.29	9.97	**65.03**	9.46
		GMM	✗	24.08	16.30	46.03	13.69
		GMM	✓	23.50	15.30	50.68	13.31
Unsupervised	Raw	Simple	✗	12.18	6.54	89.46	6.49
		Simple	✓	4.73	2.42	100.0	2.42
		GMM	✗	18.05	10.21	78.00	9.92
		GMM	✓	7.88	4.12	91.36	4.10
	CAE	Simple	✗	12.39	7.05	51.22	6.60
		Simple	✓	5.93	3.07	90.09	3.06
		GMM	✗	15.36	9.39	42.11	8.32
		GMM	✓	13.32	7.57	55.68	7.14
Supervised U-Net (oracle)				83.67	82.22	85.17	71.93

Table A.11
Assessment of crack severity estimation using number of cracks per patch (CPP), crack area per patch and maximum crack width. The table reports the mean absolute error (MAE) or mean absolute percentage error (MAPE, in %) with the ground-truth severity measure (lower is better). Best and second-best scores in bold underlined and bold, respectively.

Method		Post-processing		CPP	Area	Width
		thresh.	morph.	MAE	MAPE	MAPE (MAE px)
XAI-based – ViT-B/16	Input × Gradient	Simple	✓	1.86	1134.4	270.7 (21.89)
		GMM	✓	2.62	276.5	289.8 (20.78)
	IntGrad	Simple	✓	**1.00**	**151.9**	**194.6** (14.26)
		GMM	✓	**0.84**	201.9	268.5 (18.76)
	DeepLift	Simple	✓	2.26	529.0	252.3 (18.97)
		GMM	✓	1.89	265.7	284.6 (19.82)
	DeepLiftShap	Simple	✓	2.47	462.2	254.6 (18.86)
		GMM	✓	2.07	249.6	294.0 (19.96)
	GradientShap	Simple	✓	1.62	558.7	**200.7** (14.95)
		GMM	✓	1.16	**93.5**	210.4 (15.03)
	LRP-ViT*	Simple	✓	4.30	1351.8	394.2 (25.8)
		GMM	✓	3.35	579.9	319.8 (20.9)
Supervised U-Net (oracle)				0.74	20.1	20.8 (1.67)

A.2. Additional results with ViT-B/16 classifier

We use the LRP adaptation for ViT by [70], which outputs attribution maps of size $s \times s$ where s is the number of patch tokens. As we use ViT-B/16 with patch size 16 px and image size 224 px, the maps are of low resolution, i.e. 14×14, and upscaled to the original image resolution, which explains poor performance metrics. In real applications, a model with smaller patches (e.g., 8 px or smaller) should be used to obtain high-resolution maps, at an increased computational and memory cost (see Tables A.9–A.11).

References

[1] Y.-J. Cha, W. Choi, O. Büyüköztürk, Deep learning-based crack damage detection using convolutional neural networks, Comput.-Aided Civ. Infrastruct. Eng. 32 (5) (2017) 361–378, http://dx.doi.org/10.1111/mice.12263.

[2] C.F. Özgenel, A.G. Sorguç, Performance comparison of pretrained convolutional neural networks on crack detection in buildings, in: Proceedings of the 35th ISARC, 2018, pp. 693–700, http://dx.doi.org/10.22260/ISARC2018/0094.

[3] B.G. Pantoja-Rosero, D. Oner, M. Kozinski, R. Achanta, P. Fua, F. Perez-Cruz, K. Beyer, TOPO-loss for continuity-preserving crack detection using deep learning, Constr. Build. Mater. 344 (2022) 128264, http://dx.doi.org/10.1016/j.conbuildmat.2022.128264.

[4] L. Zhang, F. Yang, Y. Daniel Zhang, Y.J. Zhu, Road crack detection using deep convolutional neural network, in: 2016 IEEE International Conference on Image Processing, ICIP, 2016, pp. 3708–3712, http://dx.doi.org/10.1109/ICIP.2016.7533052, ISSN: 2381-8549.

[5] S. Bang, S. Park, H. Kim, H. Kim, Encoder–decoder network for pixel-level road crack detection in black-box images, Comput.-Aided Civ. Infrastruct. Eng. 34 (8) (2019) 713–727, http://dx.doi.org/10.1111/mice.12440.

[6] I. Abdel-Qader, O. Abudayyeh, M.E. Kelly, Analysis of edge-detection techniques for crack identification in bridges, J. Comput. Civ. Eng. 17 (4) (2003) 255–263, http://dx.doi.org/10.1061/(ASCE)0887-3801(2003)17:4(255).

[7] R. Zaurin, F.N. Catbas, Integration of computer imaging and sensor data for structural health monitoring of bridges, Smart Mater. Struct. 19 (1) (2009) 015019, http://dx.doi.org/10.1088/0964-1726/19/1/015019.

[8] P. Prasanna, K.J. Dana, N. Gucunski, B.B. Basily, H.M. La, R.S. Lim, H. Parvardeh, Automated crack detection on concrete bridges, IEEE Trans. Autom. Sci. Eng. 13 (2) (2016) 591–599, http://dx.doi.org/10.1109/TASE.2014.2354314.

[9] H. Xu, X. Su, Y. Wang, H. Cai, K. Cui, X. Chen, Automatic bridge crack detection using a convolutional neural network, Appl. Sci. 9 (14) (2019) 2867, http://dx.doi.org/10.3390/app9142867.

[10] C.K. Nguyen, K. Kawamura, M. Shiozaki, A. Tarighat, Development of an automatic crack inspection system for concrete tunnel lining based on computer vision technologies, IOP Conf. Ser.: Mater. Sci. Eng. 371 (1) (2018) 012015, http://dx.doi.org/10.1088/1757-899X/371/1/012015, Publisher: IOP Publishing.

[11] N.-D. Hoang, Detection of surface crack in building structures using image processing technique with an improved otsu method for image thresholding, Adv. Civ. Eng. 2018 (2018) http://dx.doi.org/10.1155/2018/3924120.

[12] S.K. Sinha, P.W. Fieguth, Neuro-fuzzy network for the classification of buried pipe defects, Autom. Constr. 15 (1) (2006) 73–83, http://dx.doi.org/10.1016/j.autcon.2005.02.005.

[13] W. Wu, Z. Liu, Y. He, Classification of defects with ensemble methods in the automated visual inspection of sewer pipes, Pattern Anal. Appl. 18 (2) (2015) 263–276, http://dx.doi.org/10.1007/s10044-013-0355-5.

[14] G. Wang, J. Xiang, Railway sleeper crack recognition based on edge detection and CNN, Smart Struct. Syst. 28 (6) (2021) 779–789, URL http://techno-press.org/content/?page=article&journal=sss&volume=28&num=6&ordernum=5. Number: 6.

[15] K. Rombach, G. Michau, K. Ratnasabapathy, L.-S. Ancu, W. Bürzle, S. Koller, O. Fink, Contrastive feature learning for railway infrastructure fault diagnostic, in: Book of Extended Abstracts for the 32nd European Safety and Reliability Conference, Research Publishing Services, 2022, pp. 1875–1881, http://dx.doi.org/10.3850/978-981-18-5183-4_S02-07-645-cd.

[16] W. Wang, W. Hu, W. Wang, X. Xu, M. Wang, Y. Shi, S. Qiu, E. Tutumluer, Automated crack severity level detection and classification for ballastless track slab using deep convolutional neural network, Autom. Constr. 124 (2021) 103484, http://dx.doi.org/10.1016/j.autcon.2020.103484.

[17] C. Koch, K. Georgieva, V. Kasireddy, B. Akinci, P. Fieguth, A review on computer vision based defect detection and condition assessment of concrete and asphalt civil infrastructure, Adv. Eng. Inform. 29 (2) (2015) 196–210, http://dx.doi.org/10.1016/j.aei.2015.01.008.

[18] B.F. Spencer, V. Hoskere, Y. Narazaki, Advances in computer vision-based civil infrastructure inspection and monitoring, Engineering 5 (2) (2019) 199–222, http://dx.doi.org/10.1016/j.eng.2018.11.030.

[19] L. Jiang, Y. Xie, T. Ren, A deep neural networks approach for pixel-level runway pavement crack segmentation using drone-captured images, 2020, http://dx.doi.org/10.48550/arXiv.2001.03257.

[20] Q. Mei, M. Gül, A cost effective solution for pavement crack inspection using cameras and deep neural networks, Constr. Build. Mater. 256 (2020) 119397, http://dx.doi.org/10.1016/j.conbuildmat.2020.119397.

[21] D. Kang, Y.-J. Cha, Autonomous UAVs for structural health monitoring using deep learning and an ultrasonic beacon system with geo-tagging, Comput.-Aided Civ. Infrastruct. Eng. 33 (10) (2018) 885–902, http://dx.doi.org/10.1111/mice.12375.

[22] A. Tatarinov, A. Rumjancevs, V. Mironovs, Assessment of cracks in pre-stressed concrete railway sleepers by ultrasonic testing, Procedia Comput. Sci. 149 (2019) 324–330, http://dx.doi.org/10.1016/j.procs.2019.01.143.

[23] A. Shayan, G.W. Quick, Microscopic features of cracked and uncracked concrete railway sleepers, Mater. J. 89 (4) (1992) 348–361, http://dx.doi.org/10.14359/2560.

[24] A.G. Carvalhido, S. Marques, F.D. Nunes, P. Correia, Automatic road pavement crack detection using SVM, 2012, URL https://api.semanticscholar.org/CorpusID:111379079.

[25] D. Kang, S.S. Benipal, D.L. Gopal, Y.-J. Cha, Hybrid pixel-level concrete crack segmentation and quantification across complex backgrounds using deep learning, Autom. Constr. 118 (2020) 103291, http://dx.doi.org/10.1016/j.autcon.2020.103291.

[26] M. Carrasco, G. Araya-Letelier, R. Velázquez, P. Visconti, Image-based automated width measurement of surface cracking, Sensors 21 (22) (2021) 7534, http://dx.doi.org/10.3390/s21227534.

[27] J. Ha, D. Kim, M. Kim, Assessing severity of road cracks using deep learning-based segmentation and detection, J. Supercomput. 78 (16) (2022) 17721–17735, http://dx.doi.org/10.1007/s11227-022-04560-x.

[28] Z. Yu, Y. Shen, Z. Sun, J. Chen, W. Gang, Cracklab: A high-precision and efficient concrete crack segmentation and quantification network, Dev. Built Environ. 12 (2022) 100088, http://dx.doi.org/10.1016/j.dibe.2022.100088.

[29] X. Yang, H. Li, Y. Yu, X. Luo, T. Huang, X. Yang, Automatic pixel-level crack detection and measurement using fully convolutional network, Comput.-Aided Civ. Infrastruct. Eng. 33 (12) (2018) 1090–1109, http://dx.doi.org/10.1111/mice.12412.

[30] Y.-C. Tsai, V. Kaul, R.M. Mersereau, Critical assessment of pavement distress segmentation methods, J. Transp. Eng. 136 (1) (2010) 11–19, http://dx.doi.org/10.1061/(ASCE)TE.1943-5436.0000051, Publisher: American Society of Civil Engineers.

[31] B.Y. Lee, J.-K. Kim, Y.Y. Kim, S.-T. Yi, A technique based on image processing for measuring cracks in the surface of concrete structures, in: Transactions, Toronto, 2007.

[32] R. Fan, M.J. Bocus, Y. Zhu, J. Jiao, L. Wang, F. Ma, S. Cheng, M. Liu, Road crack detection using deep convolutional neural network and adaptive thresholding, in: 2019 IEEE Intelligent Vehicles Symposium, IV, 2019, http://dx.doi.org/10.48550/arXiv.1904.08582, arXiv:1904.08582 [cs, eess].

[33] H. Han, H. Deng, Q. Dong, X. Gu, T. Zhang, Y. Wang, An advanced otsu method integrated with edge detection and decision tree for crack detection in highway transportation infrastructure, Adv. Mater. Sci. Eng. 2021 (2021) http://dx.doi.org/10.1155/2021/9205509.

[34] T. Yamaguchi, S. Nakamura, R. Saegusa, S. Hashimoto, Image-based crack detection for real concrete surfaces, IEEJ Trans. Electr. Electron. Eng. 3 (1) (2008) 128–135, http://dx.doi.org/10.1002/tee.20244.

[35] S. Amarasiri, M. Gunaratne, S. Sarkar, Modeling of crack depths in digital images of concrete pavements using optical reflection properties, J. Transp. Eng. 136 (6) (2010) 489–499, http://dx.doi.org/10.1061/(ASCE)TE.1943-5436.0000095.

[36] C.V. Dung, L.D. Anh, Autonomous concrete crack detection using deep fully convolutional neural network, Autom. Constr. 99 (2019) 52–58, http://dx.doi.org/10.1016/j.autcon.2018.11.028.

[37] Q. Mei, M. Gül, M.R. Azim, Densely connected deep neural network considering connectivity of pixels for automatic crack detection, Autom. Constr. 110 (2020) 103018, http://dx.doi.org/10.1016/j.autcon.2019.103018.

[38] Z. Liu, Y. Cao, Y. Wang, W. Wang, Computer vision-based concrete crack detection using U-net fully convolutional networks, Autom. Constr. 104 (2019) 129–139, http://dx.doi.org/10.1016/j.autcon.2019.04.005.

[39] S.L.H. Lau, E.K.P. Chong, X. Yang, X. Wang, Automated pavement crack segmentation using U-net-based convolutional neural network, IEEE Access 8 (2020) 114892–114899, http://dx.doi.org/10.1109/ACCESS.2020.3003638, Conference Name: IEEE Access.

[40] R. Augustauskas, A. Lipnickas, Improved pixel-level pavement-defect segmentation using a deep autoencoder, Sensors 20 (9) (2020) http://dx.doi.org/10.3390/s20092557, Number: 9 Publisher: Multidisciplinary Digital Publishing Institute.

[41] X. Sun, Y. Xie, L. Jiang, Y. Cao, B. Liu, DMA-net: DeepLab with multi-scale attention for pavement crack segmentation, IEEE Trans. Intell. Transp. Syst. 23 (10) (2022) 18392–18403, http://dx.doi.org/10.1109/TITS.2022.3158670, Conference Name: IEEE Transactions on Intelligent Transportation Systems.

[42] Y. Huang, C. Qiu, Y. Guo, X. Wang, K. Yuan, Surface defect saliency of magnetic tile, in: 2018 IEEE 14th International Conference on Automation Science and Engineering, CASE, 2018, pp. 612–617, http://dx.doi.org/10.1109/COASE.2018.8560423.

[43] M. Zhang, Y. Zhou, J. Zhao, Y. Man, B. Liu, R. Yao, A survey of semi- and weakly supervised semantic segmentation of images, Artif. Intell. Rev. 53 (6) (2020) 4259–4288, http://dx.doi.org/10.1007/s10462-019-09792-7.

[44] J.K. Chow, Z. Su, J. Wu, P.S. Tan, X. Mao, Y.H. Wang, Anomaly detection of defects on concrete structures with the convolutional autoencoder, Adv. Eng. Inform. 45 (2020) http://dx.doi.org/10.1016/j.aei.2020.101105.

[45] S. Kulkarni, S. Singh, D. Balakrishnan, S. Sharma, S. Devunuri, S.C.R. Korlapati, CrackSeg9k: A collection and benchmark for crack segmentation datasets and frameworks, 2022, http://dx.doi.org/10.48550/arXiv.2208.13054.

[46] P. Hüthwohl, R. Lu, I. Brilakis, Multi-classifier for reinforced concrete bridge defects, Autom. Constr. 105 (2019) 102824, http://dx.doi.org/10.1016/j.autcon.2019.04.019.

[47] Y.-J. Cha, W. Choi, G. Suh, S. Mahmoudkhani, O. Büyüköztürk, Autonomous structural visual inspection using region-based deep learning for detecting multiple damage types, Comput.-Aided Civ. Infrastruct. Eng. 33 (9) (2018) 731–747, http://dx.doi.org/10.1111/mice.12334.

[48] T. Dawood, Z. Zhu, T. Zayed, Computer vision–based model for moisture marks detection and recognition in subway networks, J. Comput. Civ. Eng. 32 (2) (2018) 04017079, http://dx.doi.org/10.1061/(ASCE)CP.1943-5487.0000728.

[49] E. Bianchi, M. Hebdon, Visual structural inspection datasets, Autom. Constr. 139 (2022) 104299, http://dx.doi.org/10.1016/j.autcon.2022.104299.

[50] K. Tomaszkiewicz, T. Owerko, A pre-failure narrow concrete cracks dataset for engineering structures damage classification and segmentation, Sci. Data 10 (1) (2023) 925, http://dx.doi.org/10.1038/s41597-023-02839-z, Publisher: Nature Publishing Group.

[51] A.B. Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. García, S. Gil-López, D. Molina, R. Benjamins, R. Chatila, F. Herrera, Explainable artificial intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI, 2019, http://dx.doi.org/10.48550/arXiv.1910.10045.

[52] C. Seibold, J. Künzel, A. Hilsmann, P. Eisert, From explanations to segmentation: Using explainable AI for image segmentation, in: 17th International Conference on Computer Vision Theory and Applications, VISAPP, 2022, http://dx.doi.org/10.48550/arXiv.2202.00315.

[53] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, W. Samek, On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation, PLoS One 10 (7) (2015) e0130140, http://dx.doi.org/10.1371/journal.pone.0130140.

[54] D. Baehrens, T. Schroeter, S. Harmeling, M. Kawanabe, K. Hansen, K.-R. Müller, How to explain individual classification decisions, J. Mach. Learn. Res. 11 (61) (2010) 1803–1831, URL http://jmlr.org/papers/v11/baehrens10a.html.

[55] M. Sundararajan, A. Taly, Q. Yan, Axiomatic attribution for deep networks, 2017, http://dx.doi.org/10.48550/arXiv.1703.01365, arXiv:1703.01365 [cs].

[56] A. Shrikumar, P. Greenside, A. Kundaje, Learning important features through propagating activation differences, 2019, http://dx.doi.org/10.48550/arXiv.1704.02685.

[57] S.M. Lundberg, S.-I. Lee, A unified approach to interpreting model predictions, in: Advances in Neural Information Processing Systems, Vol. 30, Curran Associates, Inc., 2017, URL https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html.

[58] M. Böhle, M. Fritz, B. Schiele, B-cos networks: Alignment is all we need for interpretability, 2022, http://dx.doi.org/10.48550/arXiv.2205.10268.

[59] S. Stalder, N. Perraudin, R. Achanta, F. Perez-Cruz, M. Volpi, What you see is what you classify: Black box attributions, 2022, http://dx.doi.org/10.48550/arXiv.2205.11266.

[60] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, A. Torralba, Learning deep features for discriminative localization, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR, 2016, pp. 2921–2929, http://dx.doi.org/10.1109/CVPR.2016.319.

[61] R.R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, D. Batra, Grad-CAM: Visual explanations from deep networks via gradient-based localization, in: 2017 IEEE International Conference on Computer Vision, ICCV, 2017, pp. 618–626, http://dx.doi.org/10.1109/ICCV.2017.74, ISSN: 2380-7504.

[62] S. Hwang, H.-E. Kim, Self-transfer learning for weakly supervised lesion localization, in: Medical Image Computing and Computer-Assisted Intervention – MICCAI 2016, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2016, pp. 239–246, http://dx.doi.org/10.1007/978-3-319-46723-8_28.

[63] F. Dubost, G. Bortsova, H. Adams, A. Ikram, W.J. Niessen, M. Vernooij, M. De Bruijne, GP-unet: Lesion detection from weak labels with a 3D regression network, in: Medical Image Computing and Computer Assisted Intervention – MICCAI 2017, in: Lecture Notes in Computer Science, Springer International Publishing, Cham, 2017, pp. 214–221, http://dx.doi.org/10.1007/978-3-319-66179-7_25.

[64] S. Chatterjee, H. Yassin, F. Dubost, A. Nürnberger, O. Speck, Weakly-supervised segmentation using inherently-explainable classification models and their application to brain tumour classification, 2022, http://dx.doi.org/10.48550/arXiv.2206.05148.

[65] G. Montavon, A. Binder, S. Lapuschkin, W. Samek, K.-R. Müller, Layer-wise relevance propagation: An overview, in: Explainable AI: Interpreting, Explaining and Visualizing Deep Learning, Vol. 11700, Springer International Publishing, Cham, 2019, pp. 193–209, http://dx.doi.org/10.1007/978-3-030-28954-6_10.

[66] A. Rezaie, R. Achanta, M. Godio, K. Beyer, Comparison of crack segmentation using digital image correlation measurements and deep learning, Constr. Build. Mater. 261 (2020) 120474, http://dx.doi.org/10.1016/j.conbuildmat.2020.120474.

[67] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, 2015, http://dx.doi.org/10.48550/arXiv.1409.1556.

[68] V. Iglovikov, A. Shvets, TernausNet: U-net with VGG11 encoder pre-trained on ImageNet for image segmentation, 2018, http://dx.doi.org/10.48550/arXiv.1801.05746.

[69] I.J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, Y. Bengio, Maxout networks, 2013, http://dx.doi.org/10.48550/arXiv.1302.4389.

[70] H. Chefer, S. Gur, L. Wolf, Transformer interpretability beyond attention visualization, in: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, IEEE, 2021, pp. 782–791, http://dx.doi.org/10.1109/CVPR46437.2021.00084.

[71] N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, O. Reblitz-Richardson, Captum: A unified and generic model interpretability library for PyTorch, 2020, URL https://github.com/pytorch/captum.

[72] C.J. Anders, D. Neumann, W. Samek, K.-R. Müller, S. Lapuschkin, Software for dataset-wide XAI: From local explanations to global insights with zennit, CoRelAy, and ViRelAy, 2023, http://dx.doi.org/10.48550/arXiv.2106.13200.

[73] J. Zhang, S.A. Bargal, Z. Lin, J. Brandt, X. Shen, S. Sclaroff, Top-down neural attention by excitation backprop, Int. J. Comput. Vis. 126 (10) (2018) 1084–1102, http://dx.doi.org/10.1007/s11263-017-1059-x.

[74] M. Böhle, M. Fritz, B. Schiele, Convolutional dynamic alignment networks for interpretable classifications, in: 2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR, 2021, pp. 10024–10033, http://dx.doi.org/10.1109/CVPR46437.2021.00990.

[75] J. Gildenblat, contributors, PyTorch library for CAM methods, 2021, URL https://github.com/jacobgil/pytorch-grad-cam.